1. Positional encoding
2. Self Attention → Multihead Self Attention
3. Layer normalization

Decoder

= { Cross Attention
= { Self mask attention

End2End flow encoder → decoder

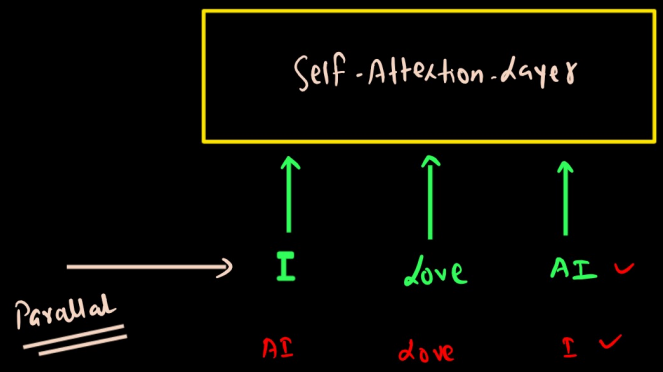
2-3

API → dam

{ FT → dam
  HF, unsloth } { coding }

Positional encoding

Problem statement → Data → Parallelly → Soln → Sequence → PE



Positional Encoding (Definition)

Positional encoding is a technique used to inject information about the position (order) of tokens into their embeddings, so that the model can understand sequence order.

Why it is needed?

Transformers process all tokens in parallel

So they don't know order by default

Example:

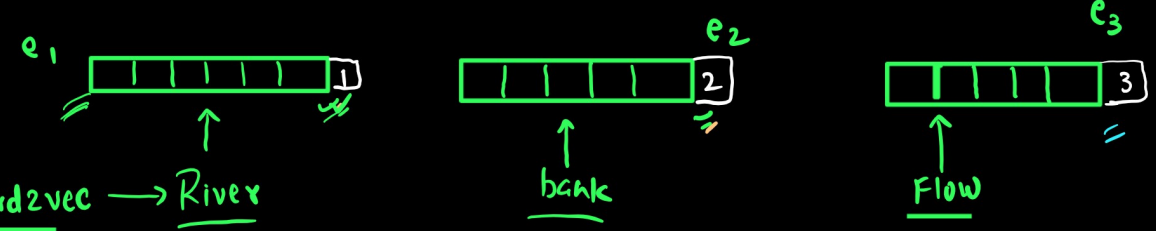
- "I love AI"
- "AI love I"

Without positional encoding → both look same to model

Dog bites man

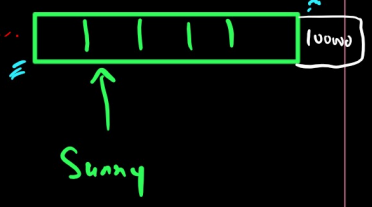
man bites dog

Self - Attention - Layer



Real time data ⟶ Self- 100000

No-limit ∞



Simple- Approach ⟶ Self- ⟶ 4 token

I Love AI Much
1 2 3 4

Self- ⟶ 4m token

I Love AI Much Again ..... You
1 2 3 4 5 ..... 4m

Self- ⟶ Limited

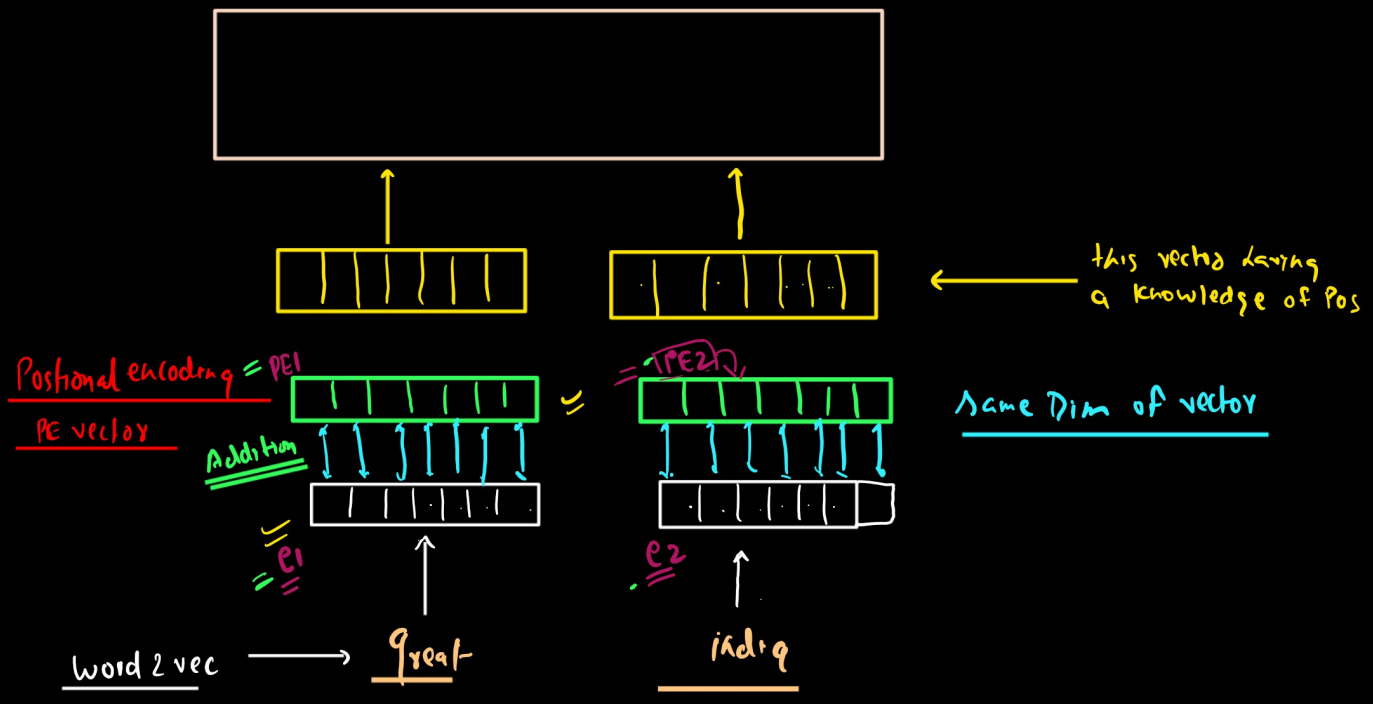
n token

1L, 10L ...

Drawback

1. No of token could be limited
2. Discreate (1, 2, 3, 4, 5 ... 100, 1000) ⟶ vector ⟶ NN

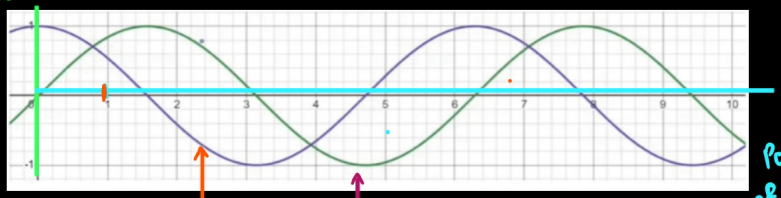
(training might become unstable)


this vector having a knowledge of Pos
Positional encoding = PE1
PE vector
= PE2
Same Dim of vector
Addition
e1
e2
Word 2 vec
great
india

Concatination
(APPEND)
Dim↑↑↑↑
10-Dim
NN / Attention
more computational Power
ADDition
e1 + PE1
same dim of vector

PE1

Encoded value



Position of the word

cos [-1 to +1]

Sin → [-1 to +1]

PE2 9-10th class

School Pythagoras

TRIGNOMETRY

Sin, cos, tan

Sin | cos

Original RP 512

= Word2vec

Great 1

India 2

Discrete ~~Continuous~~

E1 ⑥

E2 ⑥

Sin(Pos)

PE1 → Sin(1) ~ 0.84

Cos(1) ~ 0.54

Cos (Pos)

0.84 | 0.54

2

Sin (Pos)

Sin(2) ~ 0.9

Cos(2) ~ -0.4

Cos (Pos)

0.9 | -0.4

2

PE2

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{model}})$$

sin(Pos) ⟶ sin(Pos / 10000^{2i/dmodel})

cos(Pos) ⟶ cos(Pos / 10000^{2i/dmodel})

Pos ⟶ Position

dmodel ⟶ Dim of embedding model

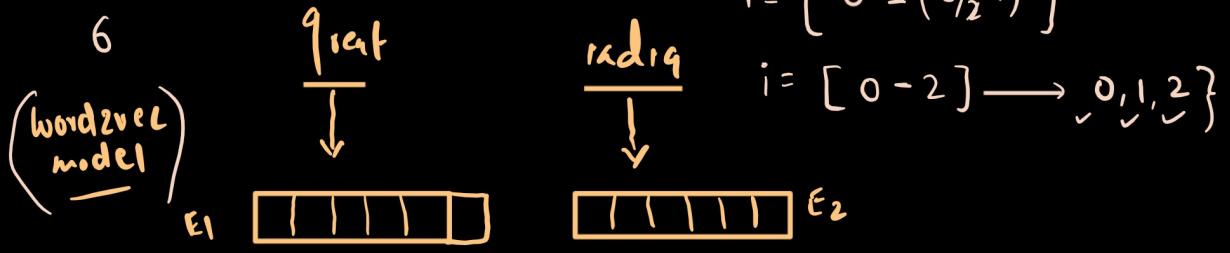
i ⟶ Range

i = [Low Range – Upper Range]

= [0 – (dmodel/2 – 1)]

i = [0 – (6/2 – 1)]

i = [0 – 2] ⟶ 0, 1, 2



4 tokens

	I	Love	my	country
Pos ⟶	0	1	2	3

Pos = 0 → great

Pos = 1 → reading

Wordvec

dmodel ⟶ 6

Dim → 6

E1 | | | | | |

E2 | | | | | |

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{model}})$$

PE2

$i = [0 - (6/2 - 1)]$

$= [0 - 2]$

$i = 0, 1, 2$ → sin/cos, cos/sin, sin/cos → 6

For i = 0 ⟶ Pos 0

$PE(0,0) = Sin(0/10000^{0}) = 0$

$PE(0,1) = Cos(0/10000^{0}) = 1$

For i = 0 → Pos → 1

$PE(1,0) = Sin(1/10000^{0}) = 0.84$

$PE(1,1) = Cos(1/10000^{0}) = 0.54$

For i = 1 (2×1)

$PE(0,2) = Sin(0/10000^{1/3}) = 0$

$PE(0,3) = Cos(0/10000^{1/3}) = 1$ (2×1+1)

For i = 1

$PE(1,2) = Sin(1/10000^{1/3}) = 0.04$

$PE(1,3) = Cos(1/10000^{1/3}) = 0.99$

For i = 2

$PE(0,4) = Sin(0/10000^{2/3}) = 0$

$PE(0,5) = Cos(0/10000^{2/3}) = 1$

For i = 2

$PE(1,4) = Sin(1/10000^{2/3}) = 0.01$

$PE(1,5) = Cos(1/10000^{2/3}) = 0.99$

PE1 | 0 | 1 | 0 | 1 | 0 | 1 |

PE2 → | 0.84 | 0.54 | 0.04 | 0.99 | 0.01 | 0.98 |

\+

Dim 6

E1

x_1

\+

E2

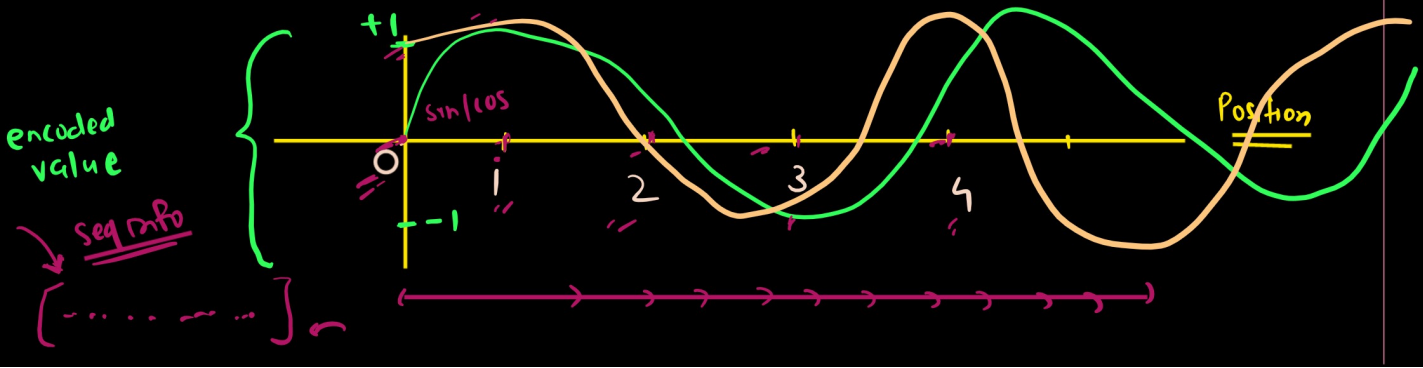
x_2

⟶ TODEY I AM TEACHING PE

~~discrete~~ Continuous (sin/cos)

token → 5

Pos ⟶ 0 1 2 3 4



(2) Self Attention ⟶ Multihead Attention

Money | bank | grow

Similarity ⟶ Dot Product

$$bank = w_1 * Money + w_2 * bank + w_3 * grow$$

(q,k)

$$[e_{bank} \cdot e_{money}]\,Money + [e_{bank} \cdot e_{bank}] * bank + [e_{bank} \cdot e_{grow}] * grow$$

(e_{bank} · e_{money}: q, k; e_{bank} · e_{bank}: q, k; e_{bank} · e_{grow}: q, k)

$$bank = \left(e_{bank} * e_{money}\right) * e_{money} + \left(e_{bank} * e_{bank}\right) * e_{bank} + \left(e_{bank} * e_{grow}\right) * e_{grow}$$

$\left(e_{bank} * e_{money}\right) = S_1$, $\left(e_{bank} * e_{bank}\right) = S_2$, $\left(e_{bank} * e_{grow}\right) = S_3$

SoftMAX

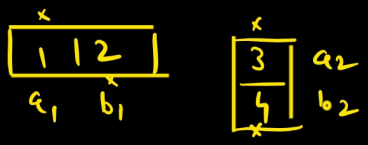
$$w_1 = \frac{e^{S_1}}{e^{S_1}+e^{S_2}+e^{S_3}} \qquad w_2 = \frac{e^{S_2}}{e^{S_1}+e^{S_2}+e^{S_3}} \qquad w_3 = \frac{e^{S_3}}{e^{S_1}+e^{S_2}+e^{S_3}}$$

$$w_1 * e_{money} + w_2 * e_{bank} + w_3 * e_{grow}$$

(value vector (v) for e_{money}; (v) for e_{bank}; (v) for e_{grow})

Self → (Q, K, V)

Contextual vector ⟵ bank



Scalar value

$(1 \times 3) + (2 \times 4) = 11$

$a_1 * a_2$ $b_1 \times b_2$

Self-Attention ⟶ Q, K, V

$D = \{a:1, b:2, c:3\}$

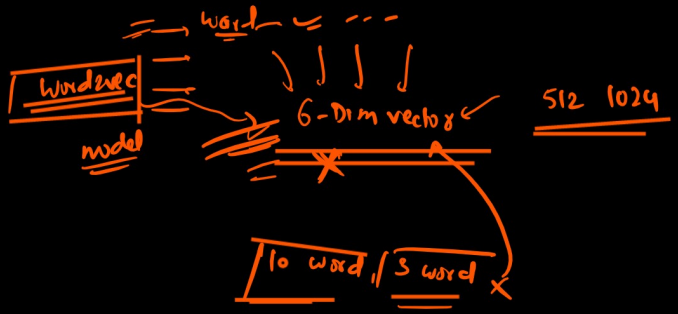
Analogy

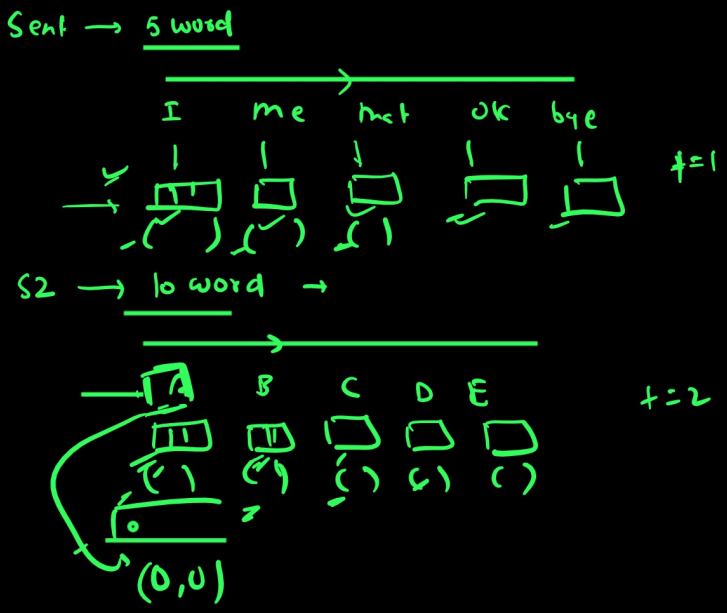
$D[q] \longrightarrow o/p = 1$

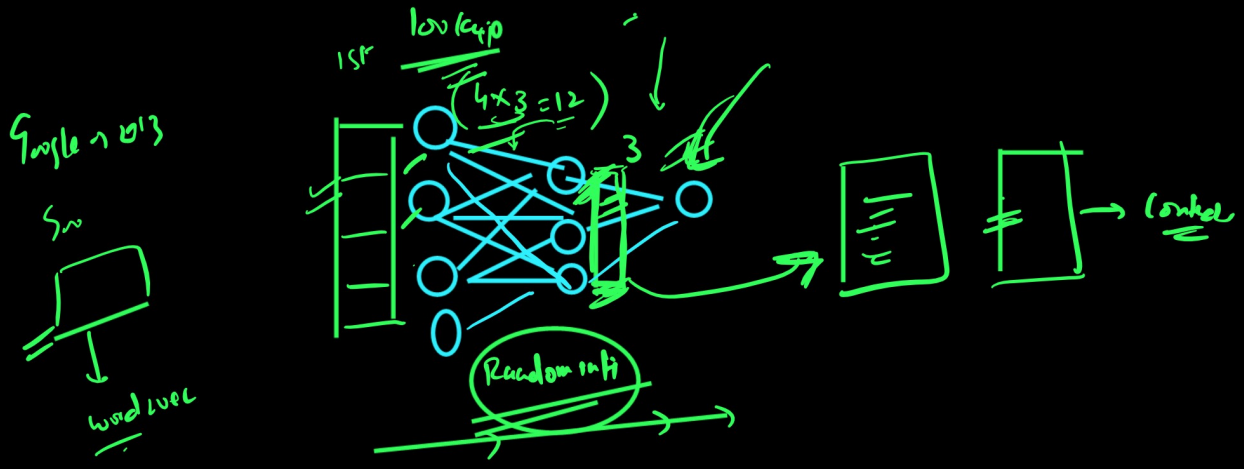
query — key — value

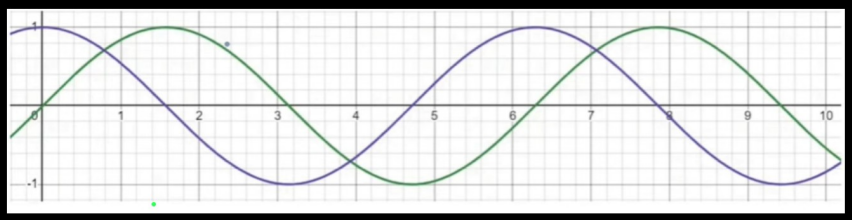
Self-Attention → extend

- multi self-attention
- cross attention
- masked " "


word
Word2vec
model
6-Dim vector
512 1024
10 word
3 word


Sent → 5 word
I
me
met
ok
bye
t=1
S2 → 10 word →
B
C
D
E
t=2
(0,0)


Google → 2013
Sm
word2vec
1st
low dim
(4x3 = 12)
3
Random init
Context



1. Encoder
2. Decoder

How the data flows in encoder

1. Positional encoding → Covered
2. Self (Attention) → multihead self Attention
3. Layer normalization
4. Neural network (FFN)
5. Residual Connection — conceptually

Data flow in decoder

- cross Attention
- Masked Attention

Overview

Pending

(Fundamental) dim Notably

- Self Attention → m, c, ms
- layer normalization
- end to end data flow in encoder-decoder

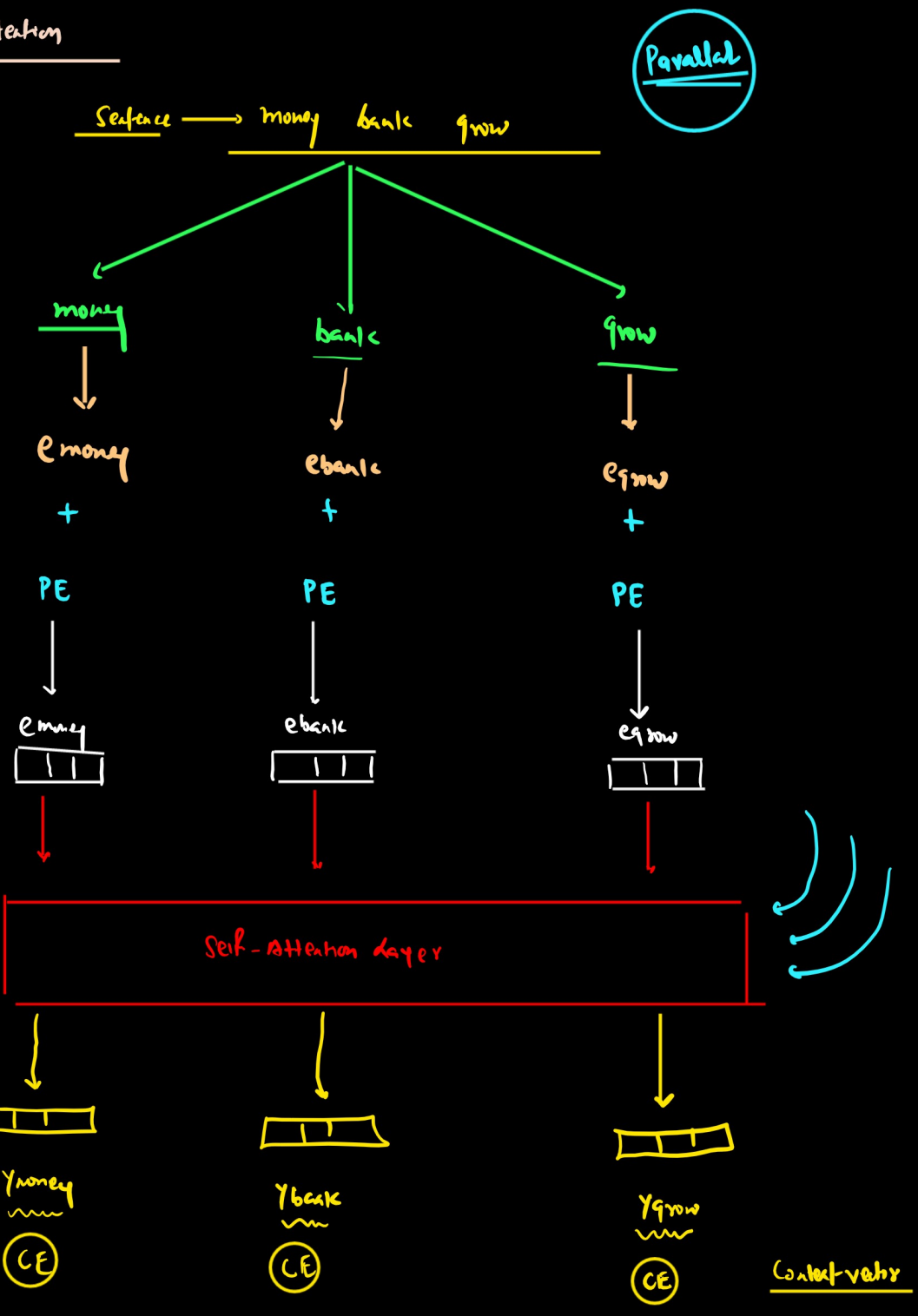
2 more 2-3

= Core

APR, PT, RAC, Agent-, Proj

Self Attention

Parallel



Similarity score

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Self-Attention → Contextual vector

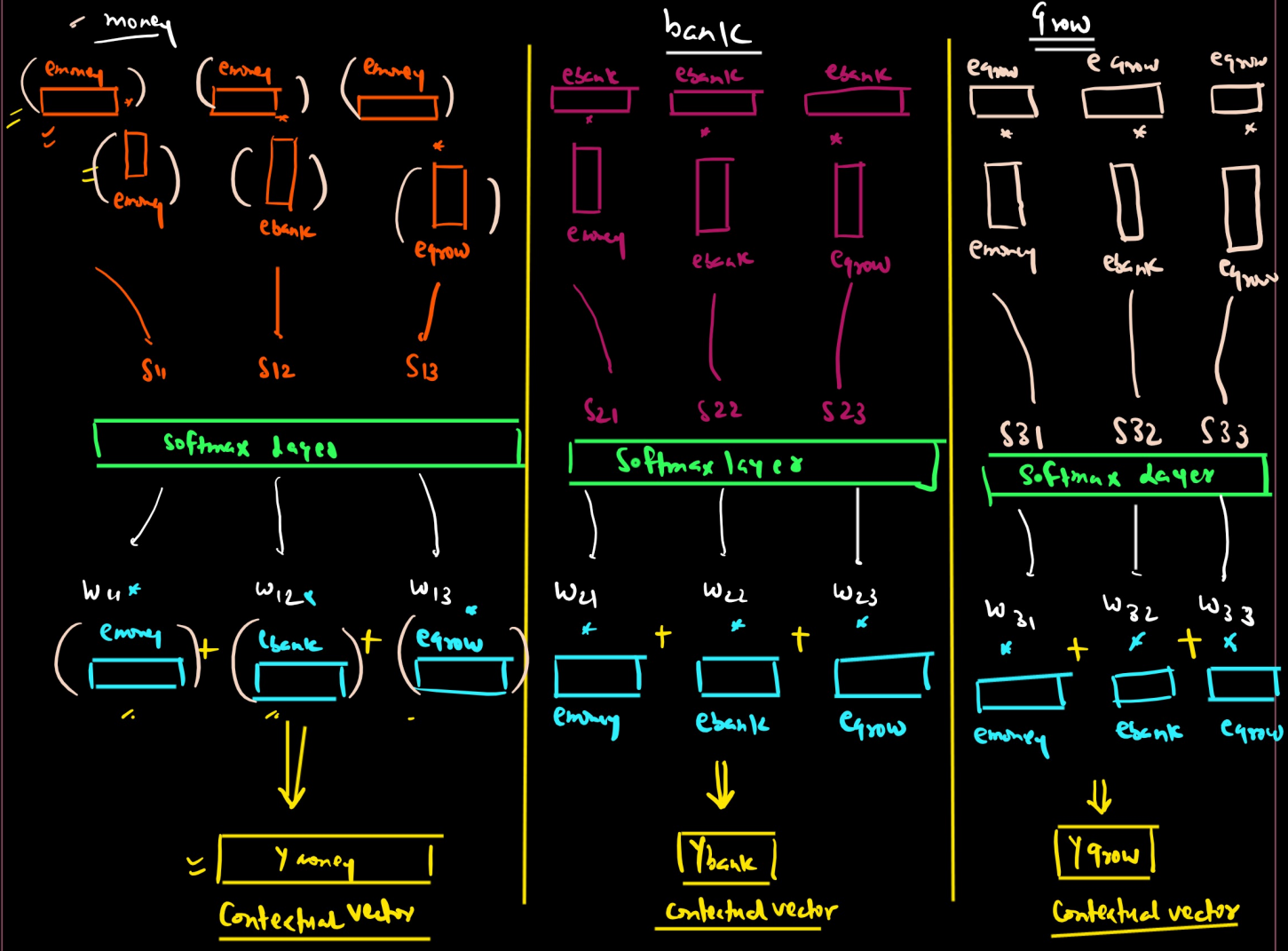
money bank grow ⟶ How this text is being processed using Self Attention

Find out the Relation of each word with word

- money CE = $W_{11} * e_{money} + W_{12} * e_{bank} + W_{13} * e_{grow}$ (similarity score)
- bank CE = $(W_{21}) * e_{money} + (W_{22}) * e_{bank} + (W_{23}) * e_{grow}$
- grow CE = $W_{31} * e_{money} + W_{32} * e_{bank} + W_{33} * e_{grow}$

SS

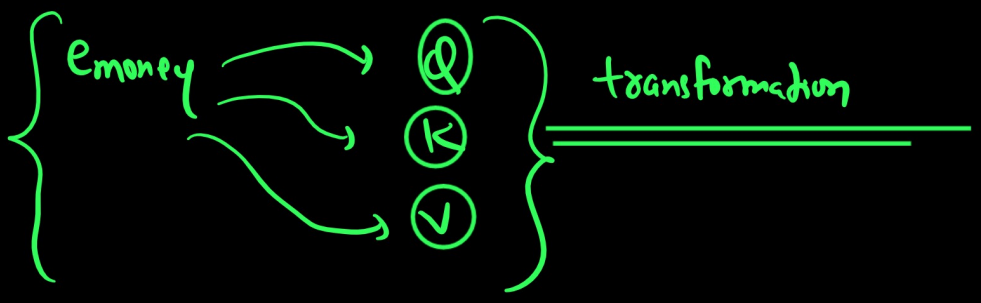
Dot product ↑

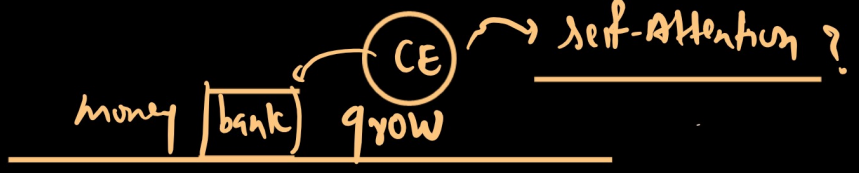


Same vector ⟶ (multiple times)

(this is confusing)

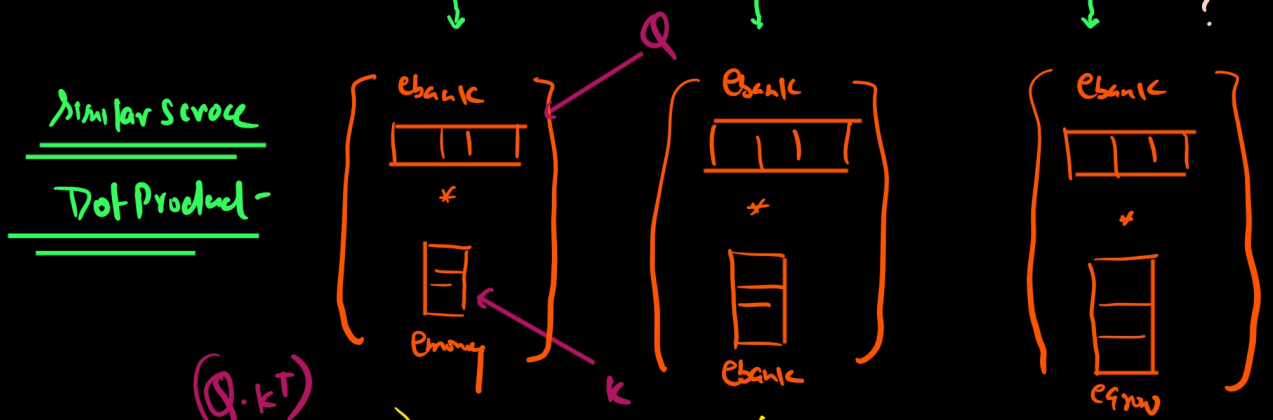
← Q, K, V

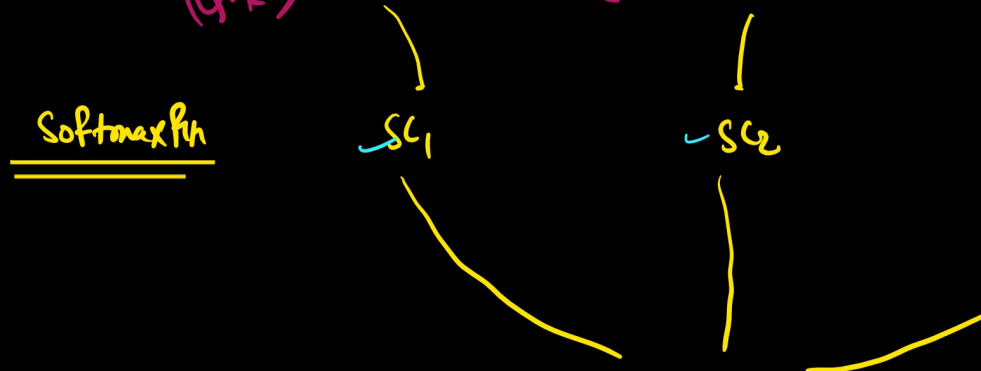




map this word in context of other word

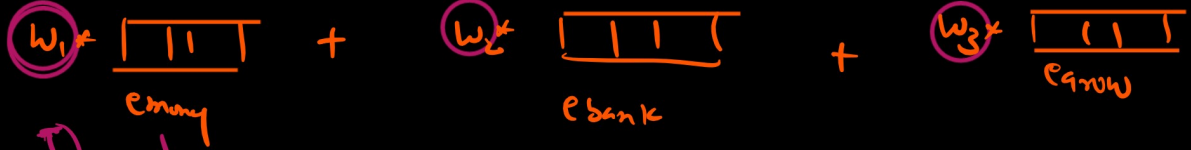
$$e_{bank}^{CE} = W_{11} * e_{money} + W_{12} * e_{bank} + W_{13} * e_{grow}$$



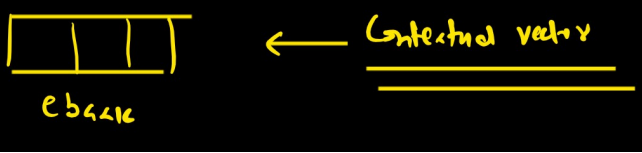


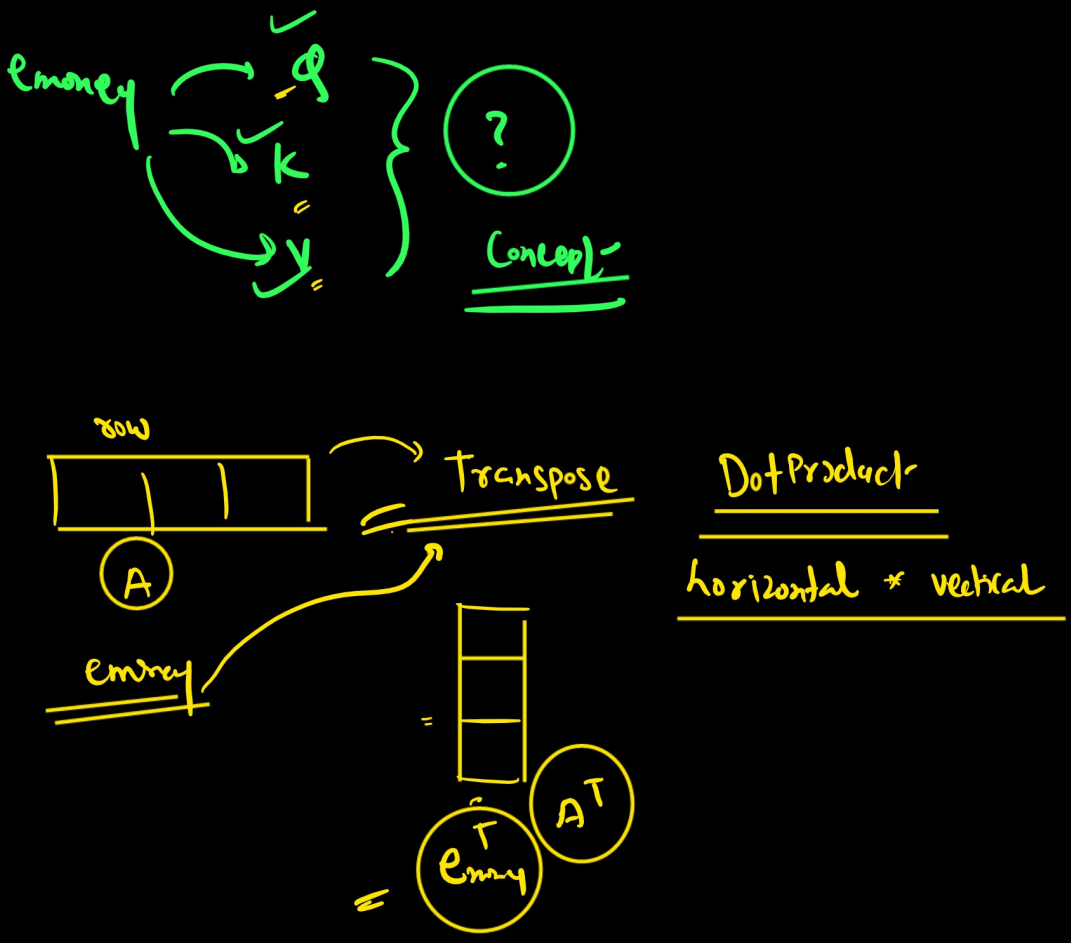
$$\text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_K}}\right) * V$$ Value

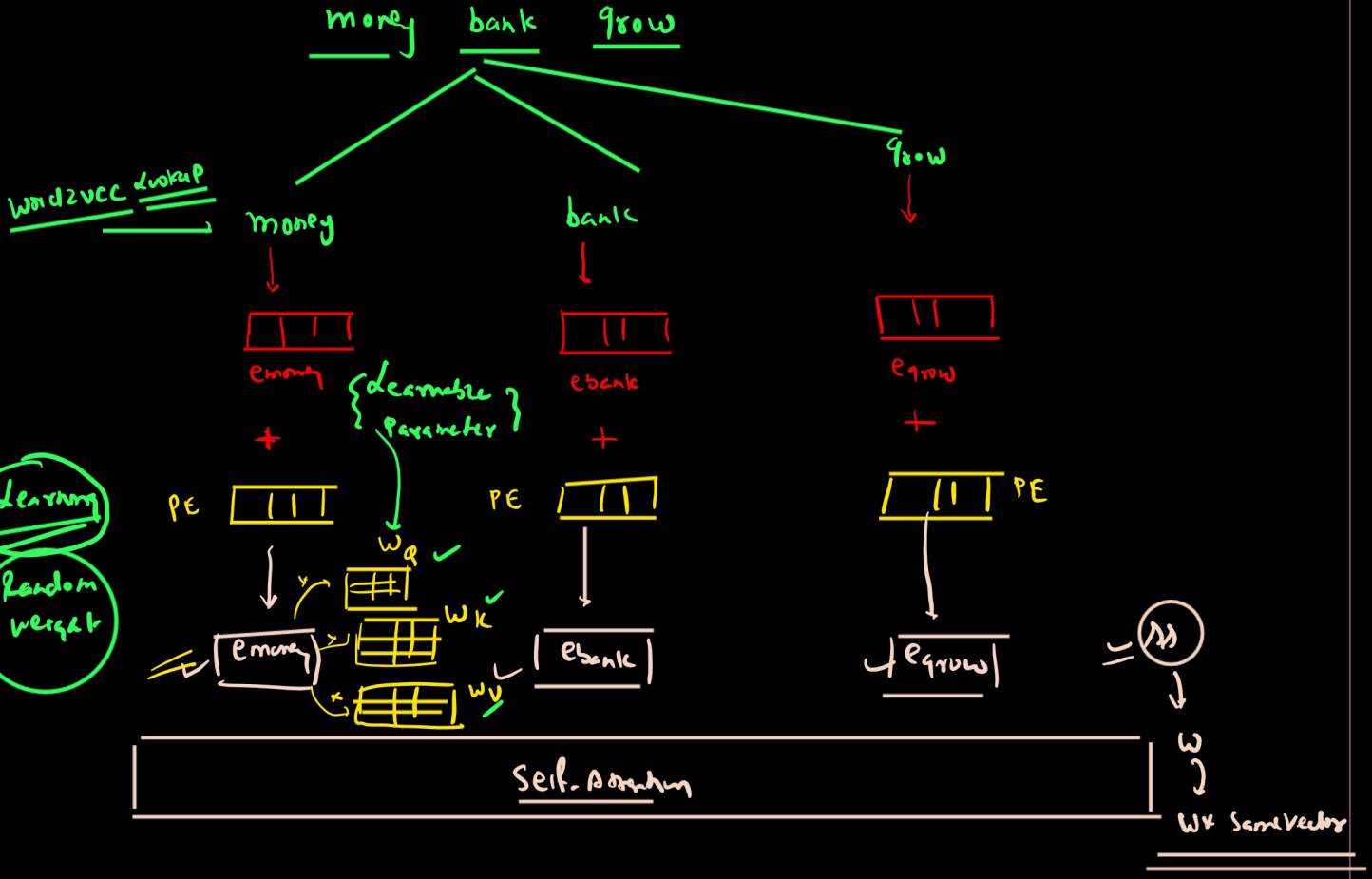
$$w_1 = \frac{e^{sc_1}}{e^{sc_1} + e^{sc_2} + e^{sc_3}} \qquad w_2 = \frac{e^{sc_2}}{e^{sc_1} + e^{sc_2} + e^{sc_3}} \qquad w_3 = \frac{e^{sc_3}}{e^{sc_1} + e^{sc_2} + e^{sc_3}}$$

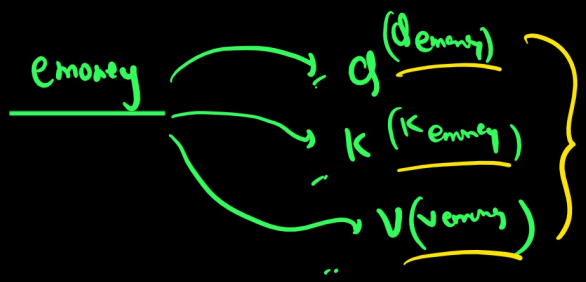


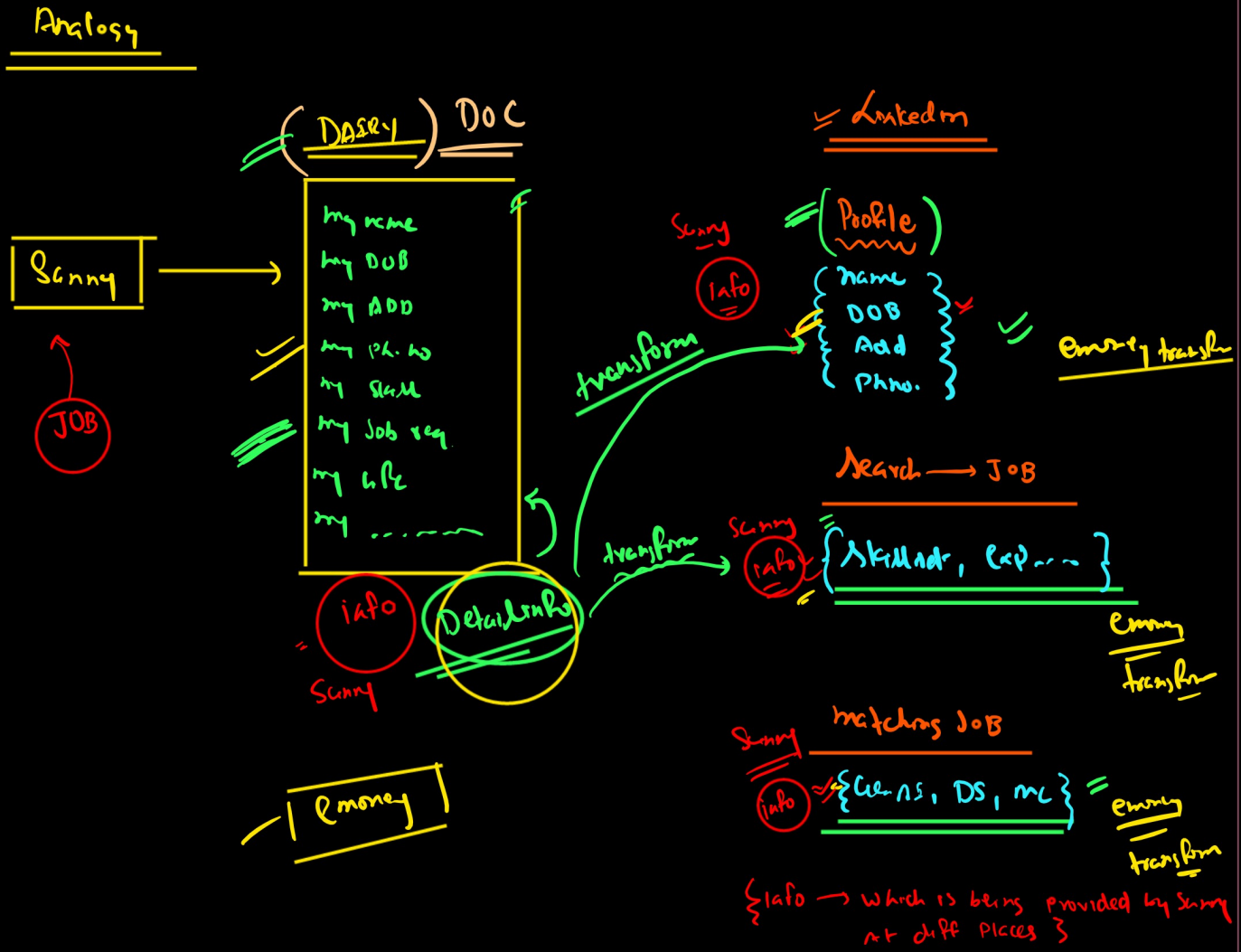
$$\text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_K}}\right) * V$$

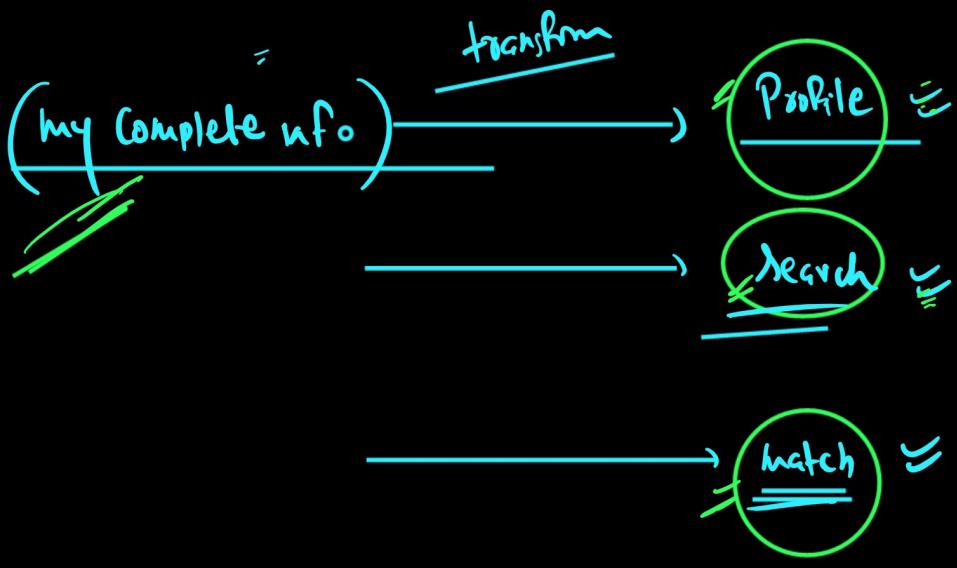


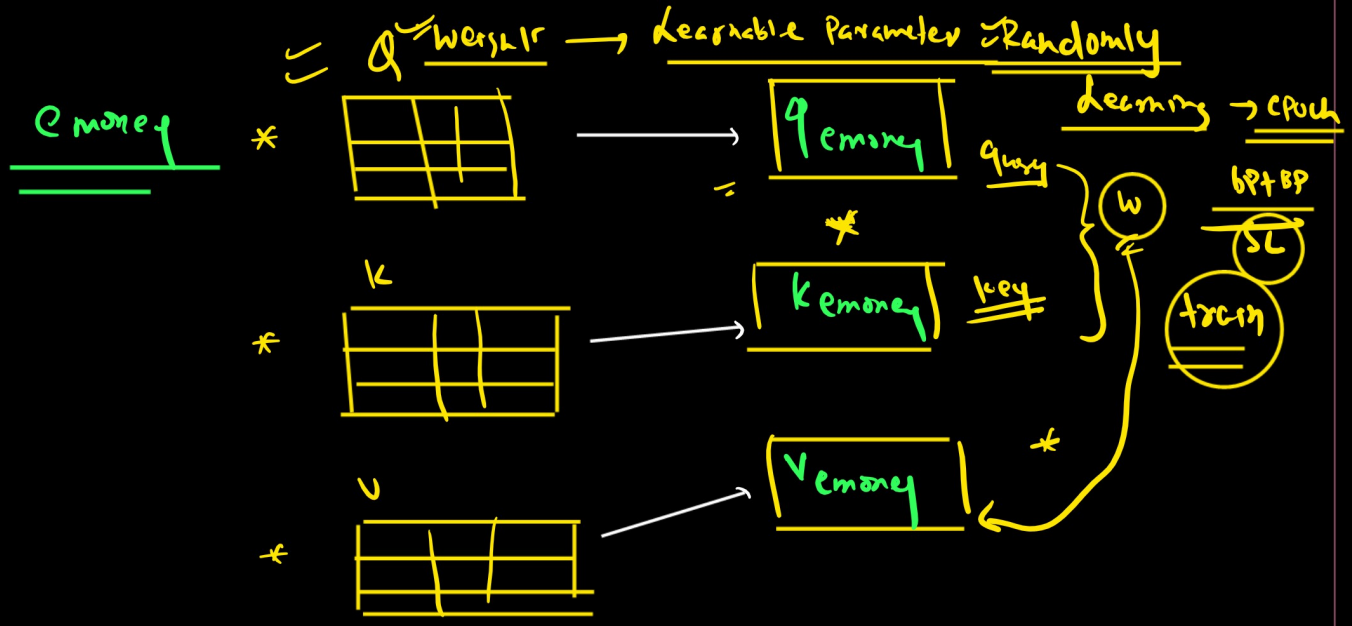

emoney
q
k
v
?
Concept
row
Transpose
A
emoney
=
A^T
e^T_{money}
=
DotProduct
horizontal * vertical

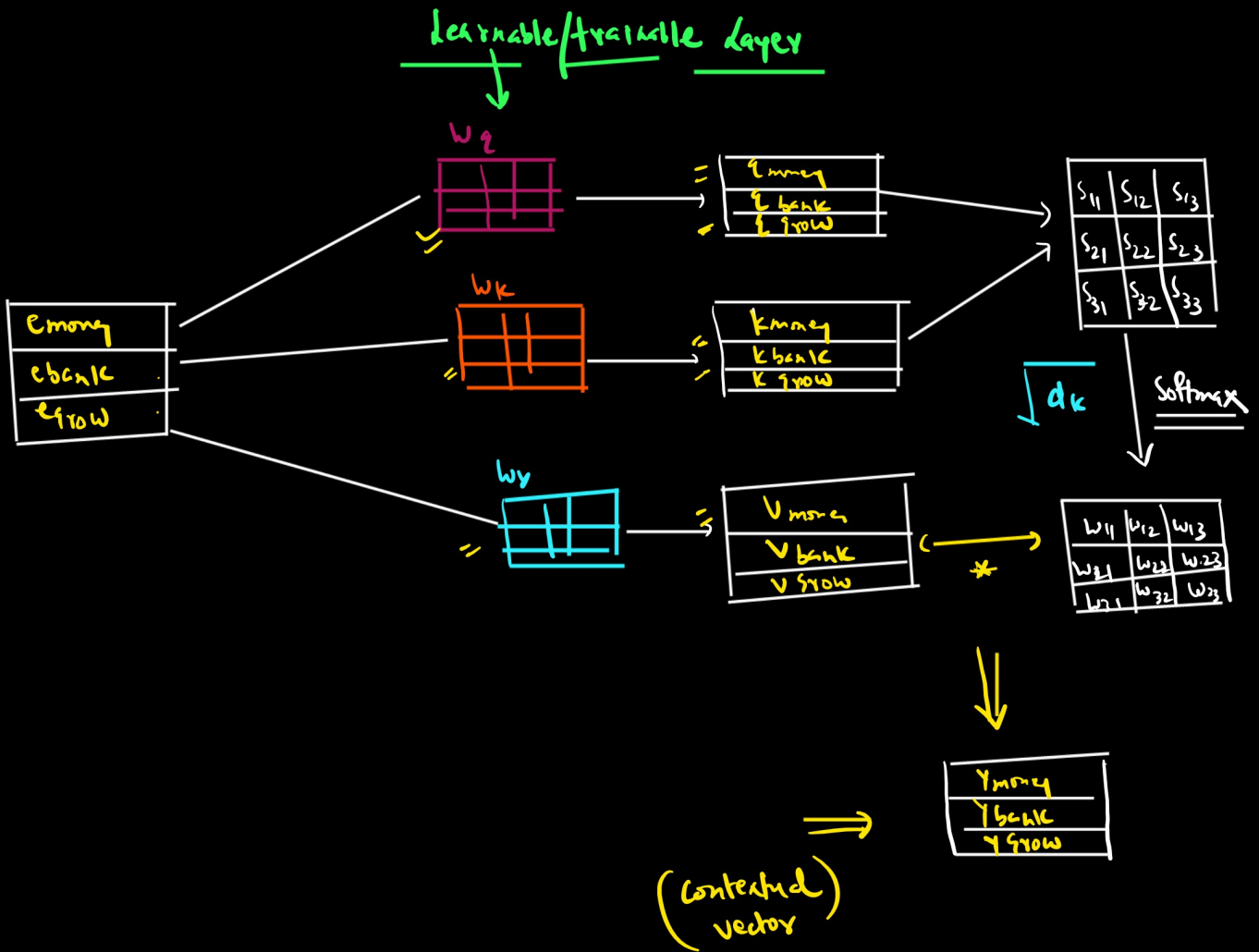

money
bank
grow
word2vec lookup
money
bank
grow
emoney
ebank
egrow
{learnable parameter}
+
+
+
PE
PE
PE
learning
Random weight
W_q
W_K
W_V
emoney
ebank
egrow
Self-Attention
W
W* same vector


emoney
q (qemoney)
k (kemoney)
v (vemoney)


Analogy
Doc
Sunny
JOB
my name
my DOB
my Add
my Ph. no
my Skill
my Job req.
my life
my .....
info
Sunny
Details info
transform
Linkedin
Profile
Sunny
info
name
DOB
Add
Phno.
emoney transform
Search → JOB
transform
Sunny
info
{Skillset, Exp....}
emoney transform
emoney
Sunny
matching JOB
info
{Clients, DS, mc}
emoney transform
{info → which is being provided by Sunny at diff places}


transform
my Complete info
Profile
Search
match

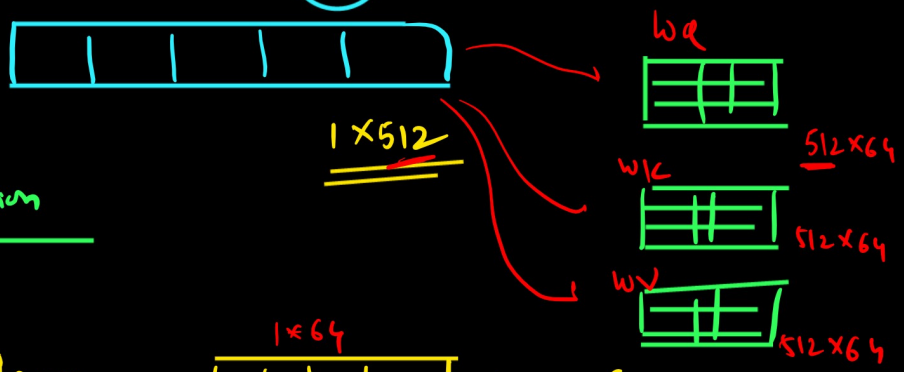

q = weight → Learnable Parameter : Randomly
learning → epoch
emony
qemony
kemony
vemony
query
key
w
bp+bp
sc
train


learnable/trainable layer
Wq
Wk
Wv
emony
ebank
erow
qmony
qbank
qrow
kmony
kbank
krow
vmony
vbank
vrow
dk
Softmax
Ymony
Ybank
Yrow
(contextual vector)

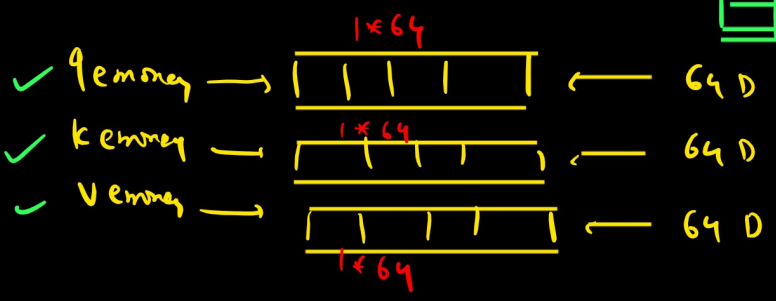
money

↓

emoney (512)



basic matrix operation



(64)

qemoney * kemoney = value ?

↓

✗

Softmax

$\left(\frac{\text{value}}{\sqrt{d_k}}\right) = (\text{value}'')$

$\sqrt{64} = 8$

↓

$\left(\frac{\text{normalized}}{\text{value}}\right)$

dimension

= k | q | v

multihead self attention / decoder masked attention

tomorrow

512 * 64

512 × 100

512 x

= You can take anything

1×100 (1×64)

$q * w_q$

$k * w_k$

$v * w_v$

= Final summary find doc

Complex

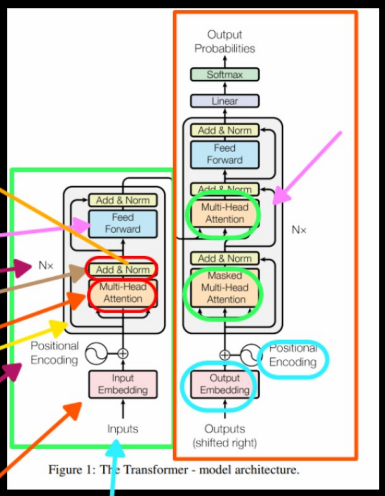


Figure 1: The Transformer - model architecture.

= (LLM → Self. attention)

machine translation

= (Encoder ↕ decoder)

Complete

Addition ~ (Add)

Concatination ~ (appending)

= Feed forward network → NN = DL

= Fundamental of AI model

30 mins → multihead Self Attention

→ (layer normalization) ≈ decoding parameter

normalization → [↓ ↓ ↓ ↓ ↓ ↓ ↓]

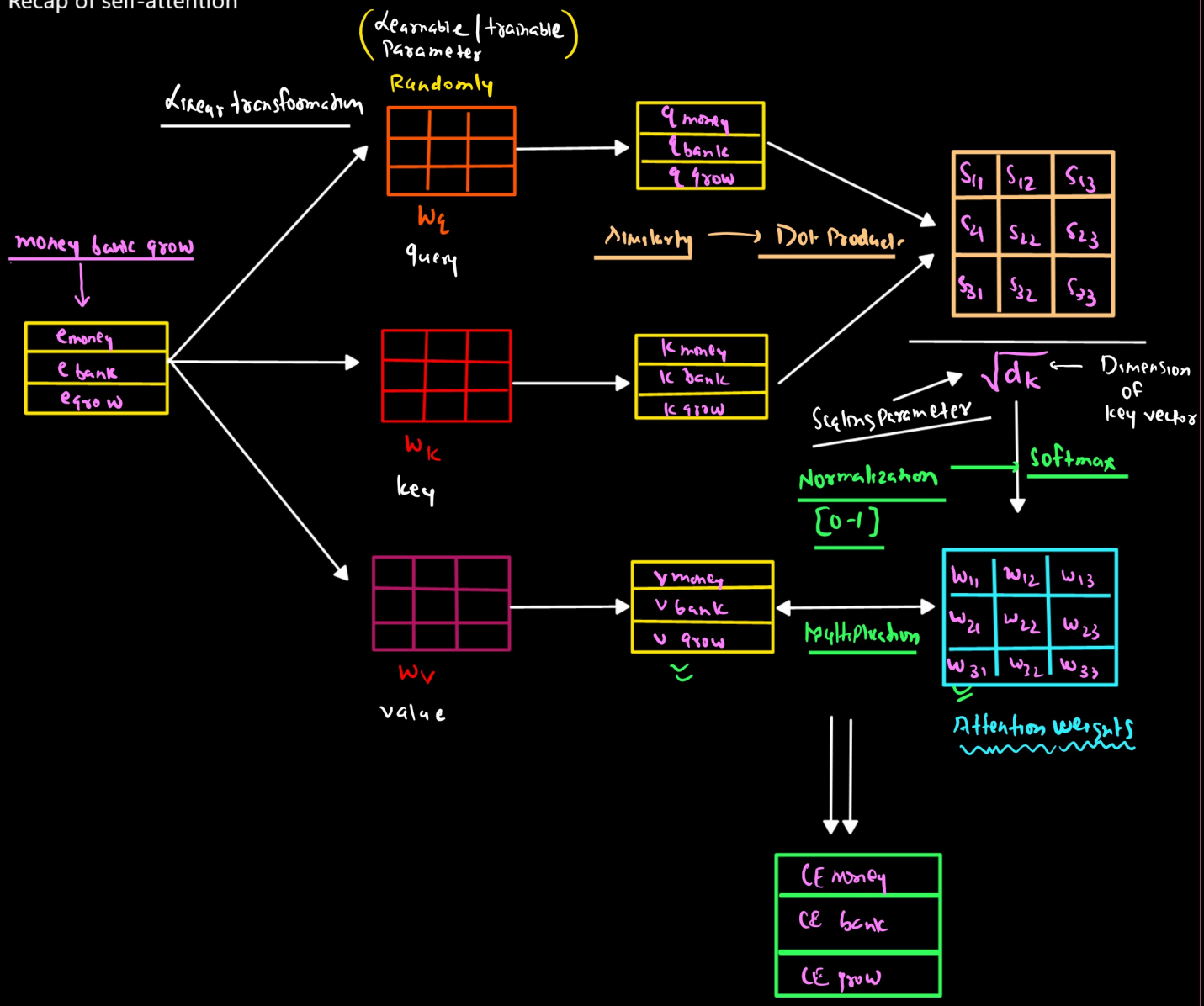
Transformer training is parallel, but inference is sequential due to autoregressive generation.

Parameter

(Weight) ← Attention

NN

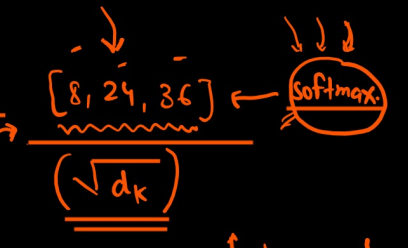
Recap of self-attention

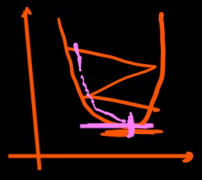


$\sqrt{d_k}$ → Dimension of the key vector, used to scale attention scores for stability.

What it achieves
Prevents extremely large values
Keeps softmax smooth
Better gradient flow
Stable training

Softmax → Converts that scaled attention scores into probabilities (weights) that sum to 1.





$$\frac{[8, 24, 32]}{\sqrt{d_k}} = \text{Softmax}([\cdot\ \cdot\ \cdot]) \rightarrow [0-1]$$

Query (Q) → What I am looking for

Key (K) → What I contain / what I offer

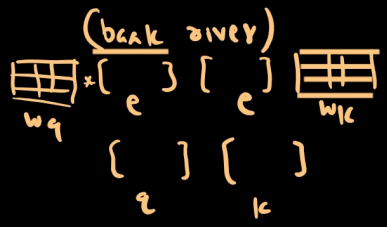
Value (V) → Actual information I provide

Query vs Key → Find relevance

Score=(Q.K)

This computes:

How relevant is this word to me



Softmax → Convert to weights

weights=softmax(QK/sqrt(dk))

Gives importance:

high score → high attention

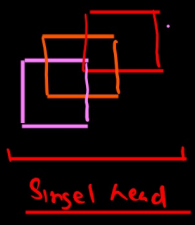
low score → low attention

Multiply with Value → Get final meaning

Output=weights· V

This gives: contextual representation

Multiheaded Self Attention



A head is an self-attention mechanism defined by its WQ, WK, WV projection matrices.

One head means single set of Wk, Wq, Wv matrix

Multihead means multiple set of Wk, Wq, Wv matrix

Single-head = one viewpoint

Multi-head = multiple viewpoints at the same time

Deeper context

multiple perspective

Multi-head self-attention is used because a sentence can have multiple perspectives, and using multiple heads allows the model to capture different relationships and meanings in parallel.

Why Multi-Head Self-Attention?

1 Used for ambiguous sentences (NLP)

Seat- Bank

Many sentences have multiple meanings / interpretations

2 Limitation of single-head attention

Deep context

Single-head attention can capture only one perspective

3 Reality in NLP

A sentence can have multiple perspectives simultaneously

Original Research Paper

8 Heads

Head → hyperparameter

multiple perspective

Deep context

One head → one set of Q,K,V

multihead → multiple set of Q,K,V

Example Sentences (Ambiguity)
"The chicken is ready to eat"
Can mean:
Chicken is cooked (to be eaten)
Chicken is hungry (ready to eat food)

"Man saw the astronomer with a telescope"
Ambiguity:
Who has the telescope? Man or astronomer?

Why Multi-Head Helps?
Multi-head attention allows the model to:
Capture multiple relationships
Understand different meanings simultaneously
Focus on different parts of the sentence in parallel

Transformer

multiple self attention

Deeper perspective

Paragraph

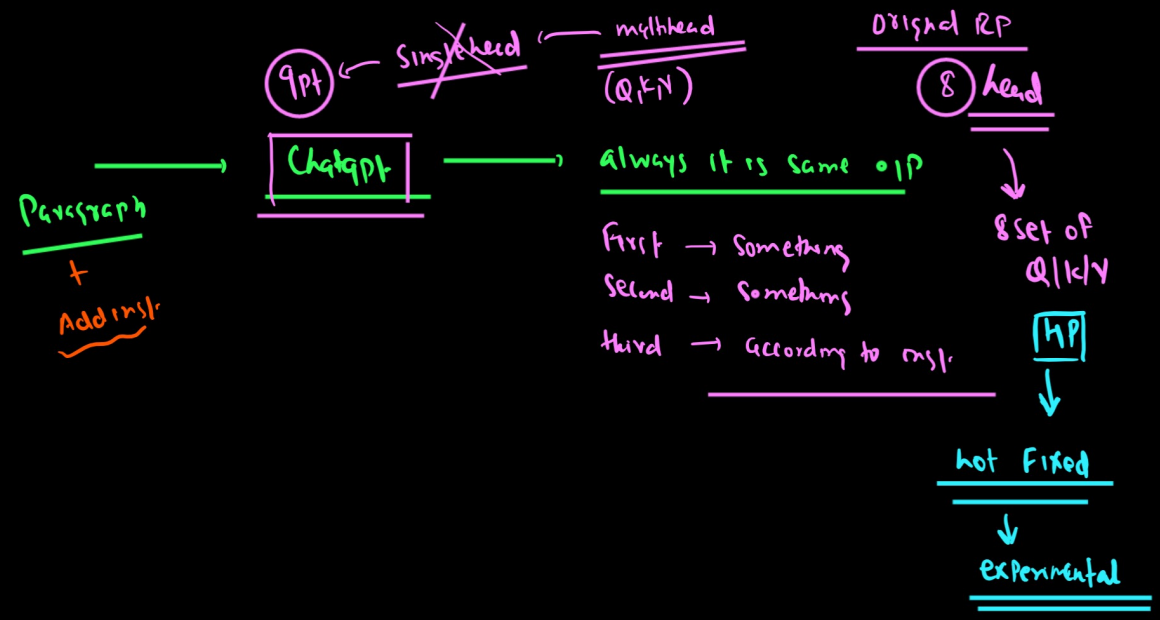
In electrical engineering, a transformer is a passive component that transfers electrical energy from one electrical circuit to another circuit, or multiple circuits. A varying current in any coil of the transformer produces a varying magnetic flux in the transformer's core, which induces a varying electromotive force (EMF) across any other coils wound around the same core. Electrical energy can be transferred between separate coils without a metallic (conductive) connection between the two circuits. Faraday's law of induction, discovered in 1831, describes the induced voltage effect in any coil due to changing magnetic flux.

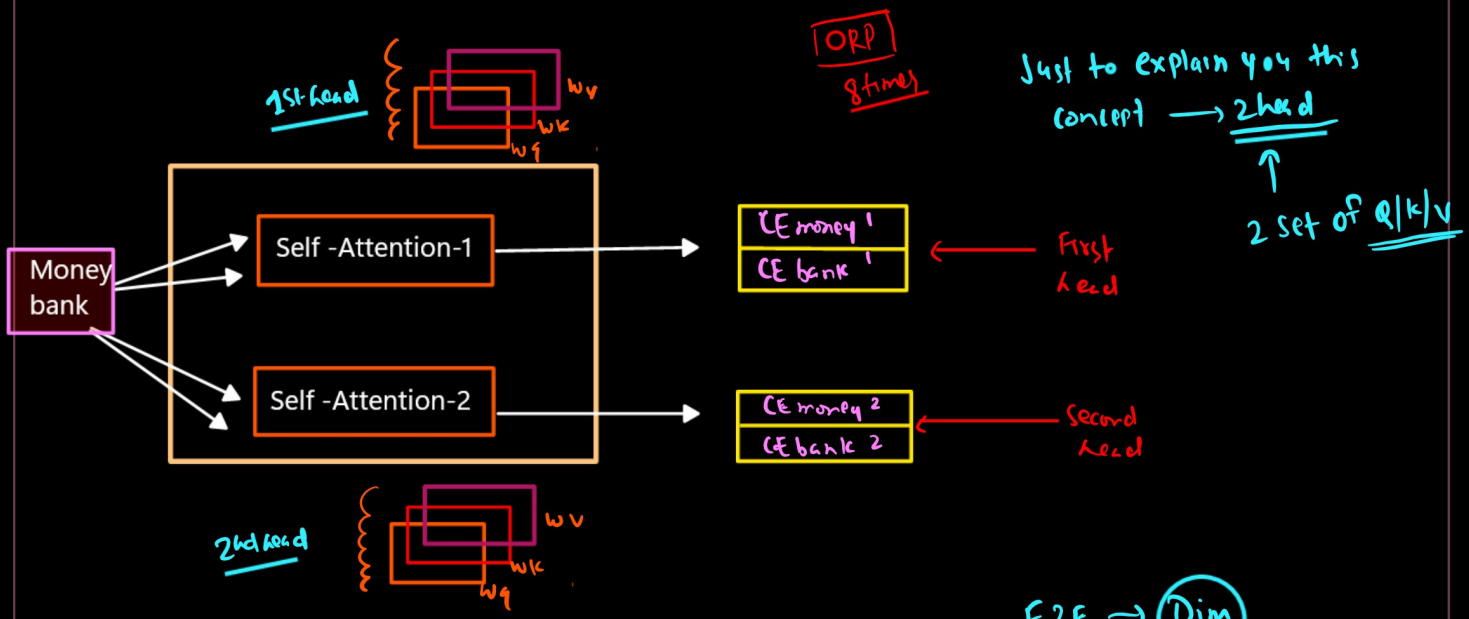
Summary 1 — one perspective

A transformer is a passive electrical component that transfers energy between circuits via electromagnetic induction. A varying current in one coil induces a magnetic flux in the core, generating an electromotive force (EMF) in other coils. Faraday's law of induction explains this phenomenon, enabling energy transfer without direct conductive connection.

Summary 2 ← another perspective

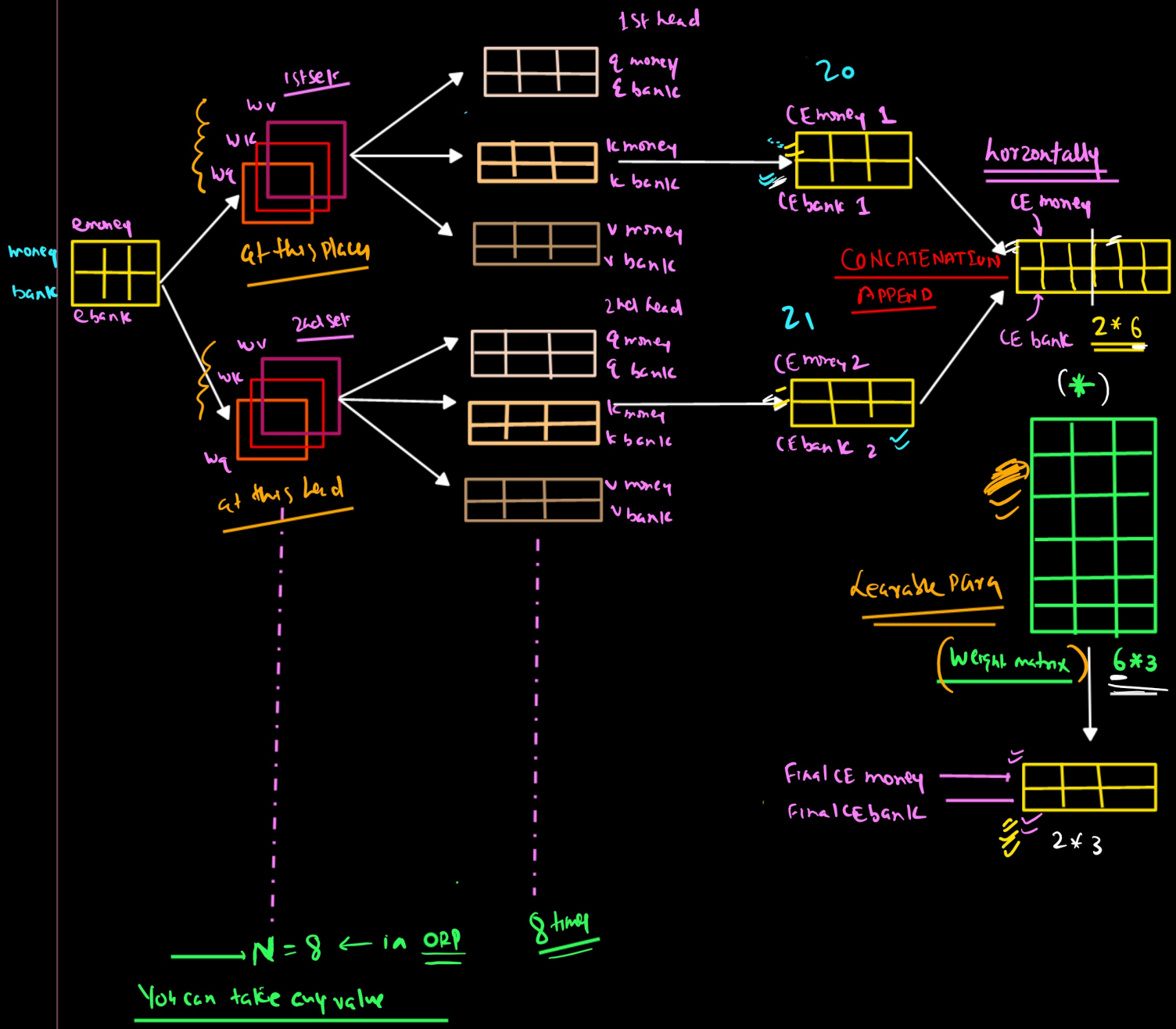
Transformers use electromagnetic induction to transfer electrical energy between circuits. A changing current in one coil creates a magnetic flux, inducing voltage in another coil. This process, based on Faraday's law, allows energy transfer without physical connections.

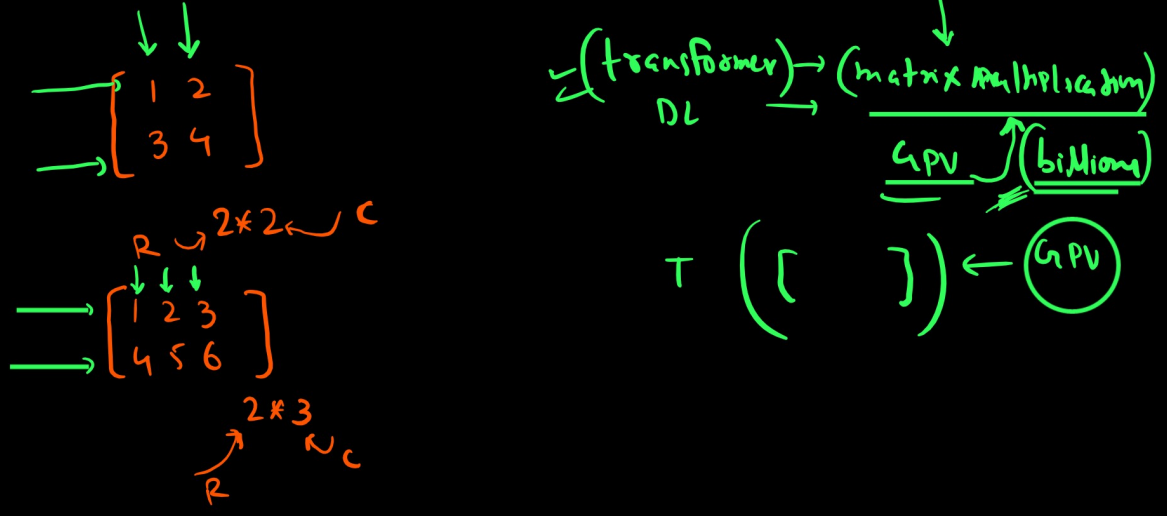




E2E → Dim

One head means single set of Wk, Wq, Wv matrix
Transformer original paper having 8 heads
but it is hyperparameter you can have as many as head





$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3\times 1}$$

Col matrix

vector

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix}_{1*3}$$

Row matrix

vector

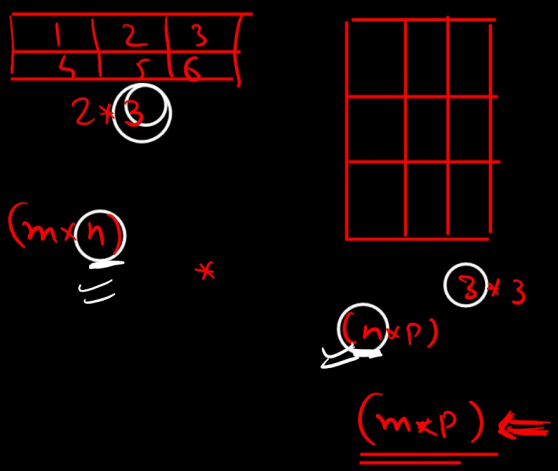
matrix multiplication

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} 1*3+2*5 & 1\times 4+2\times 6 \\ 3\times 3+4\times 5 & 3*4+4\times 6 \end{bmatrix}$$

$2\times 2 \qquad 2\times 2$

$$= \begin{bmatrix} 13 & 16 \\ 29 & 36 \end{bmatrix}$$

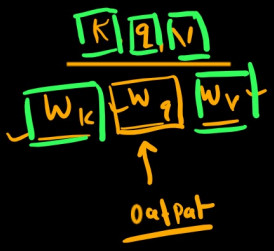
(2×2)



Cross-Attention

Cross-attention is a mechanism where one sequence attends to another different sequence to gather relevant information.

Cross-attention is a mechanism in Transformer architectures where the model uses queries from the output sequence and keys and values from the input sequence to focus on relevant parts of the input while generating the output.



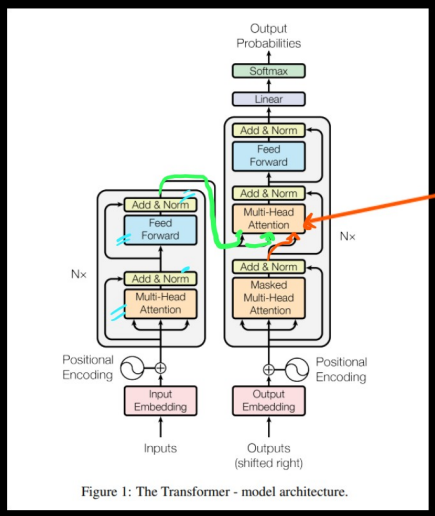
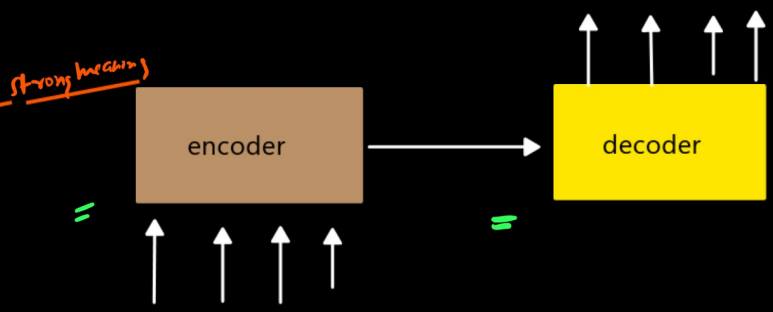
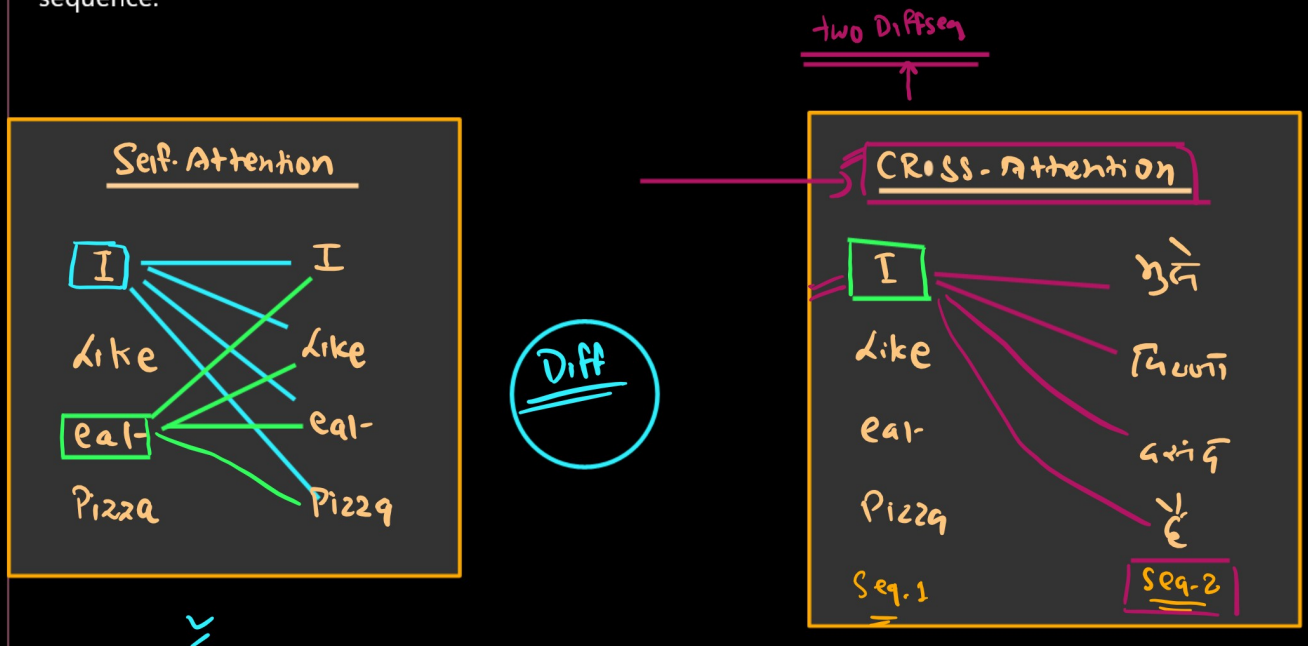


Figure 1: The Transformer - model architecture.



Self-attention looks within the same sentence, while cross-attention allows one sequence to attend to another sequence.



Same sequence check the imp of word wrt the other word

Cross-Attention

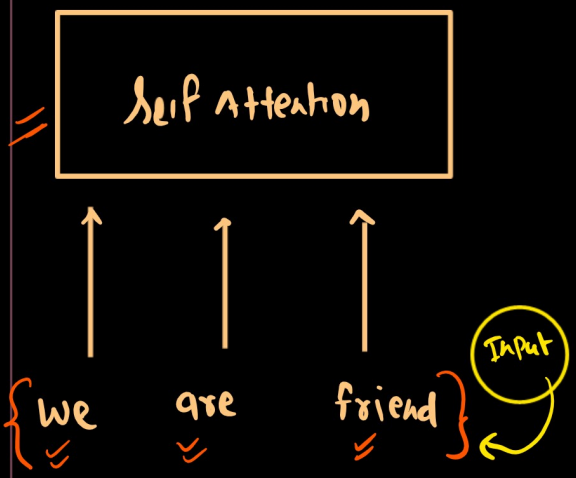
Check imp of the word from one seq. to the another seq.

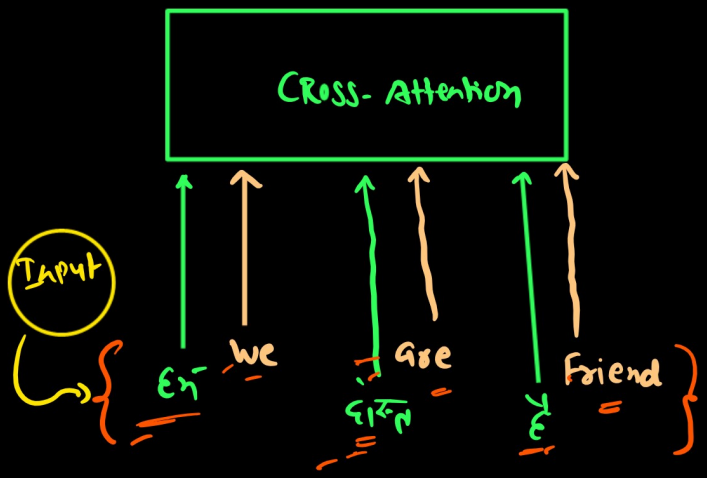
Self-Attention vs Cross Attention

→ The input
→ The processing
→ The output

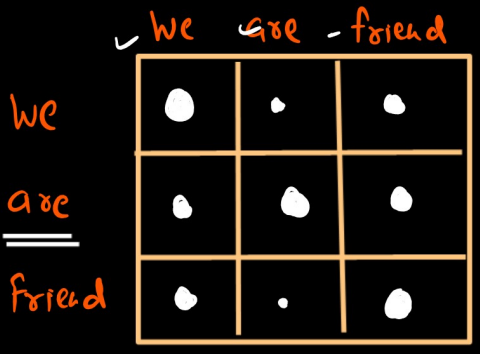
Encoder side

Decoder side



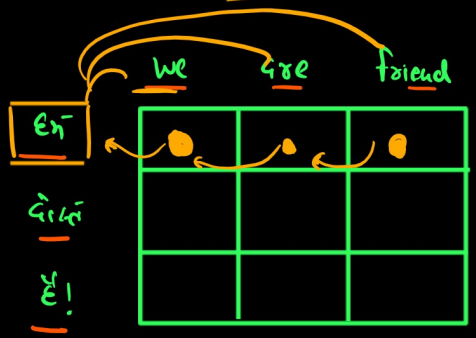


Visual Representation

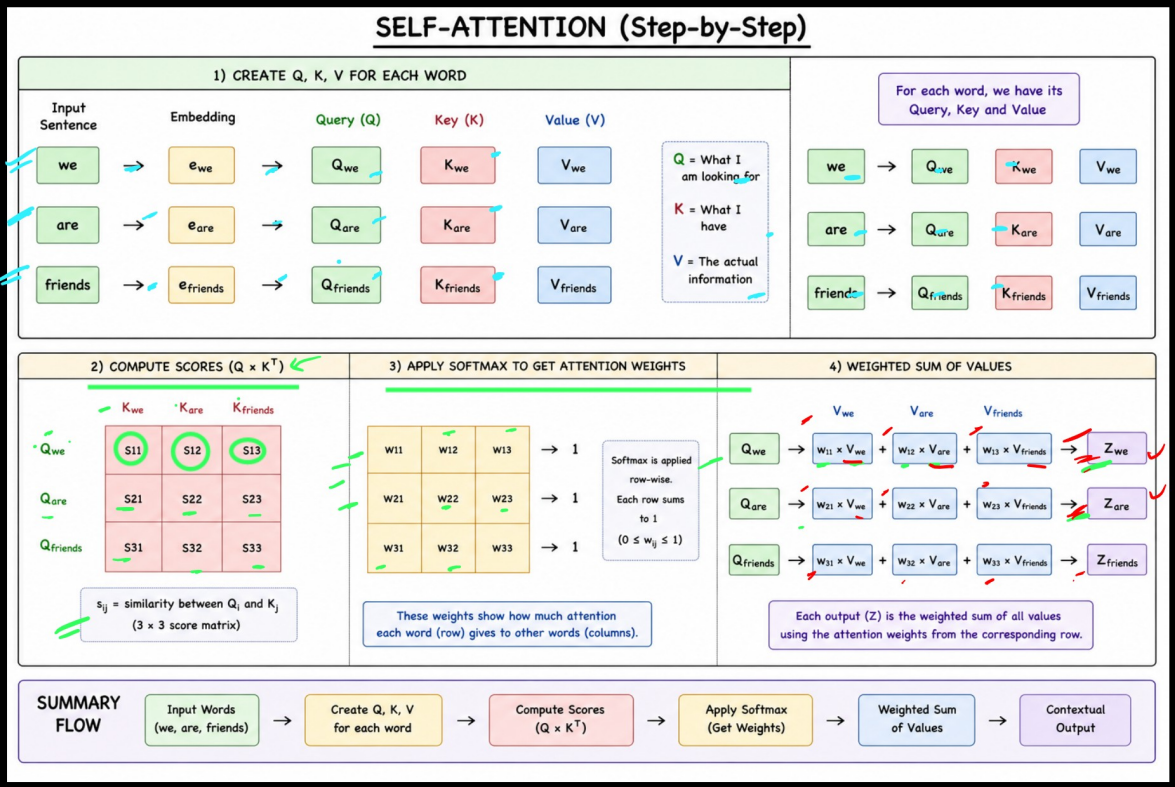


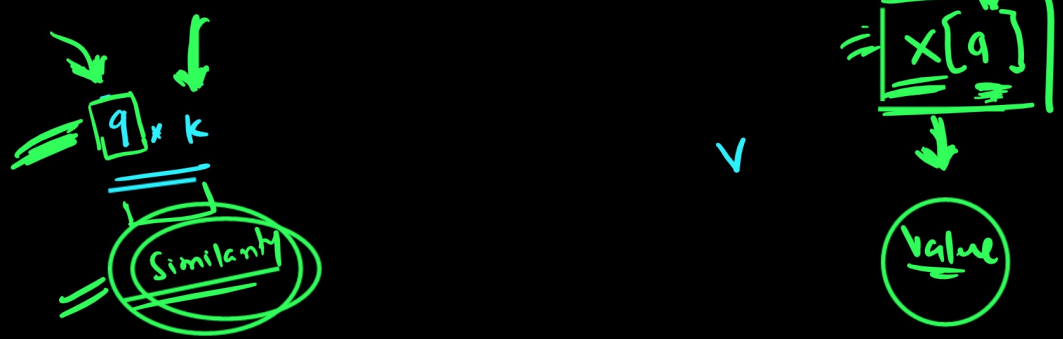
Same seq.

Cross. Attention Visual Repreat

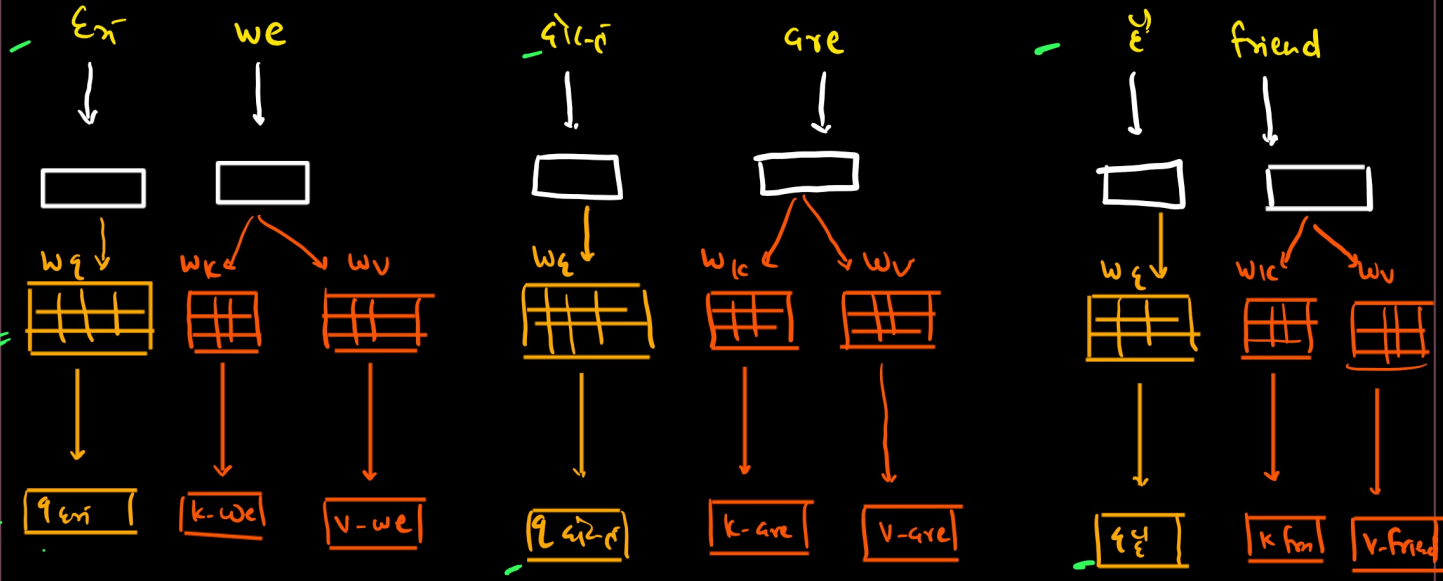


Diff-seq

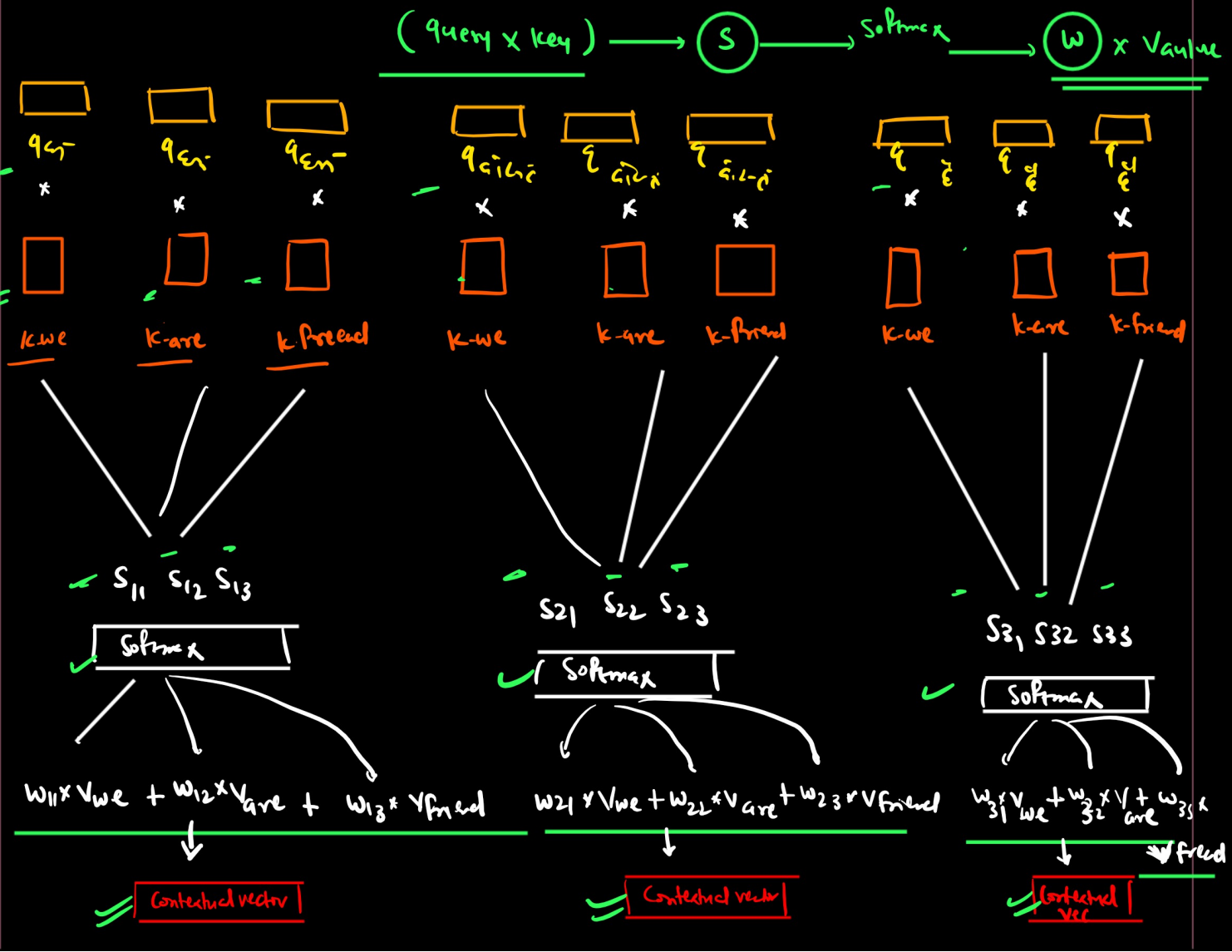

SELF-ATTENTION (Step-by-Step)
1) CREATE Q, K, V FOR EACH WORD
Input Sentence
Embedding
Query (Q)
Key (K)
Value (V)
we
are
friends
Q = What I am looking for
K = What I have
V = The actual information
For each word, we have its Query, Key and Value
2) COMPUTE SCORES (Q × K^T)
3) APPLY SOFTMAX TO GET ATTENTION WEIGHTS
4) WEIGHTED SUM OF VALUES
Softmax is applied row-wise. Each row sums to 1 (0 ≤ w_ij ≤ 1)
s_ij = similarity between Q_i and K_j (3 × 3 score matrix)
These weights show how much attention each word (row) gives to other words (columns).
Each output (Z) is the weighted sum of all values using the attention weights from the corresponding row.
SUMMARY FLOW
Input Words (we, are, friends)
Create Q, K, V for each word
Compute Scores (Q × K^T)
Apply Softmax (Get Weights)
Weighted Sum of Values
Contextual Output

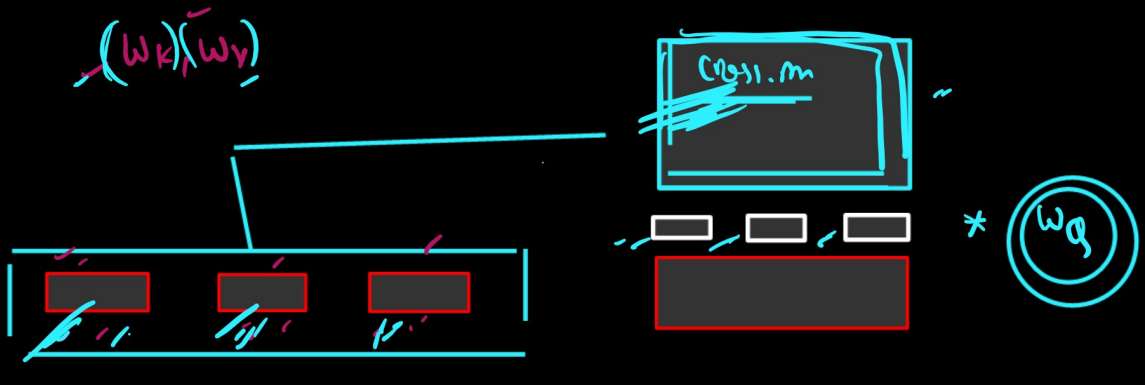

Dict :
x = { "a":1, "b":2 }
x[q]
q * k
Similarity
V
value

Vector Vector
key, value
IIP
query vector
OIP



Cross-attention is a mechanism in Transformer architectures where the model uses queries from the output sequence and keys and values from the input sequence to focus on relevant parts of the input while generating the output.




$(W_K)(W_V)$
W_Q

Masked Self-Attention (Definition)

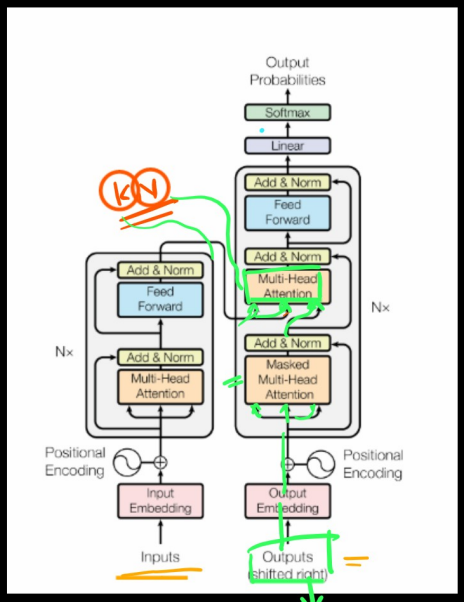
Masked self-attention is a type of self-attention where future tokens are blocked so that each token can only attend to itself and previous tokens.

Supervised training

we are friend | ક્યાં છો તું

I/P → O/P Supervisor

Label data



we are friend | ક્યાં છો તું ←

masked self-attention

we are friend

$$\text{ક્યાં CE} = w_{11} \times V_{\text{ક્યાં}} + w_{12} \times V_{\text{છો}} + w_{13} \times V_{\text{તું}}$$

$$\text{છો CE} = w_{21} \times V_{\text{ક્યાં}} + w_{22} \times V_{\text{છો}} + w_{23} \times V_{\text{તું}}$$

$$\text{તું CE} = w_{31} \times V_{\text{ક્યાં}} + w_{32} \times V_{\text{છો}} + w_{33} \times V_{\text{તું}}$$

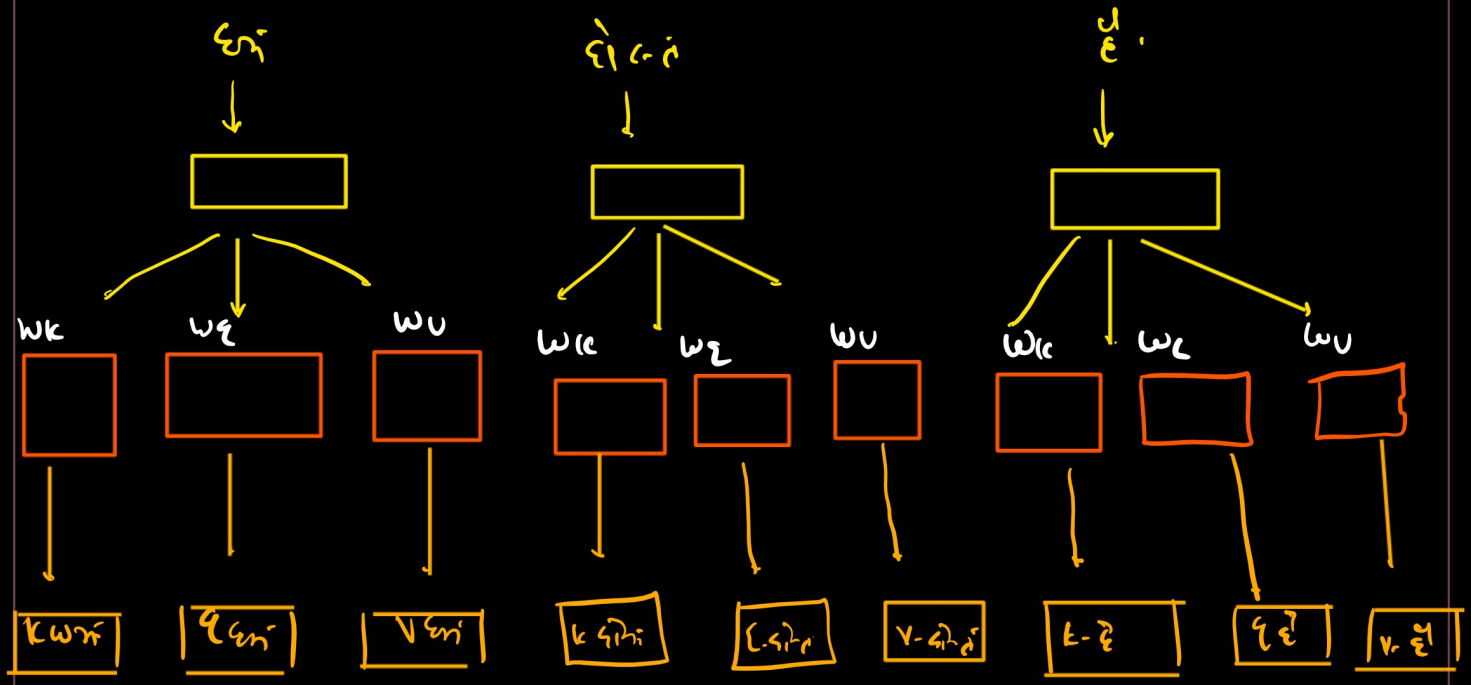
training - Parallel

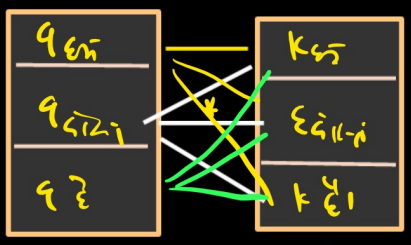
Decoder → Parallelly

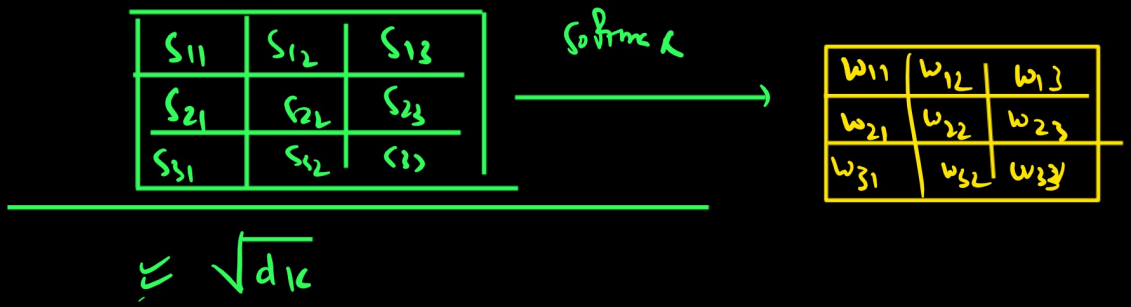
MASKED SELF Attention

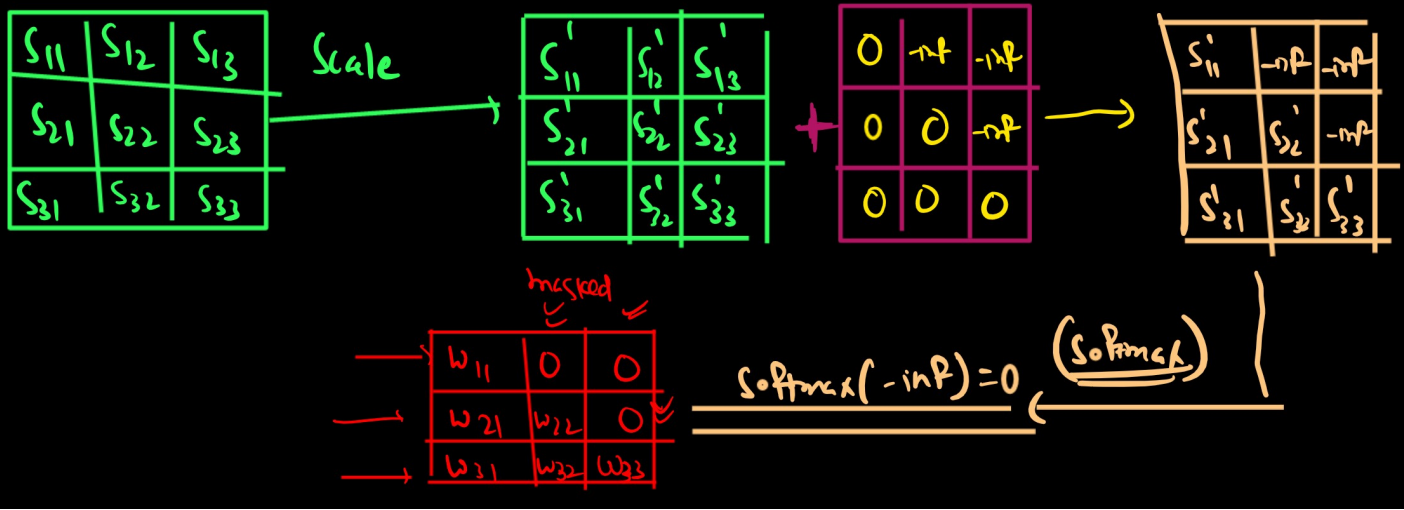
1. Data decrease
2. Overfitting ← training ↑↑↑ Acc

testing ↓↓↓ Acc






Softmax
$\frac{}{\sqrt{d_k}}$


Scale
+
masked
Softmax(-inf) = 0
(Softmax)

Layer normalization standardizes the features of each vector to improve training stability and performance.

LayerNorm normalizes a vector and scales and shifts it using learnable parameters.

Layer Normalization Formula

$$\text{LayerNorm}(x) = \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} \cdot \gamma + \beta$$

Std Scalar

Advanced Formula (transformer)

Meaning of each term
- x → input vector (e.g., token embedding)
- μ → mean of the vector
- σ^2 → variance
- ε → small value (avoid division by zero)
- γ → learnable scaling parameter
- β → learnable shift parameter

CE money

Norm

ε → small value

$\frac{x-\mu}{\sqrt{\sigma^2+\epsilon}} * \gamma + \beta$

$\sqrt{\sigma^2+\epsilon}$ small

zero

x=[0.8, −1.2, 2.4]

-1.2 0 0.8 2.4 (Scattered)

Step 1: Compute Mean

$$\mu = \frac{0.8 + (-1.2) + 2.4}{3} = \frac{2.0}{3} \approx 0.67$$

μ

$\frac{8}{0} = \infty$

dividing by zero error

Step 2: Compute Variance

$$\sigma^2 = \frac{(0.8-0.67)^2 + (-1.2-0.67)^2 + (2.4-0.67)^2}{3}$$

$$= \frac{(0.13)^2 + (-1.87)^2 + (1.73)^2}{3} = \frac{0.0169 + 3.4969 + 2.9929}{3} = \frac{6.5067}{3} \approx 2.17$$

Stats → variance, Stddev

Step 3: Normalize

$$x_{norm} = \frac{x-\mu}{\sqrt{\sigma^2}}$$

$$\sqrt{2.17} \approx 1.47$$

Now:
- $(0.8 - 0.67)/1.47 \approx 0.09$
- $(-1.2 - 0.67)/1.47 \approx -1.27$
- $(2.4 - 0.67)/1.47 \approx 1.18$

normalize

[0.09, -1.27, 1.18] ← Compress

→ γ → scale parameter

→ β → shift parameter

Step 4: Scale & Shift

$$y = x_{norm} \cdot \gamma + \beta$$

γ ← γ - learning_rate * gradient

β ← β - learning_rate * gradient

BP

DL

γ = [1, 1, 1], β = [0, 0, 0] (during learning) optimizer

[0.09, -1.27, 1.18] = training

optimizer BP

gradient decent

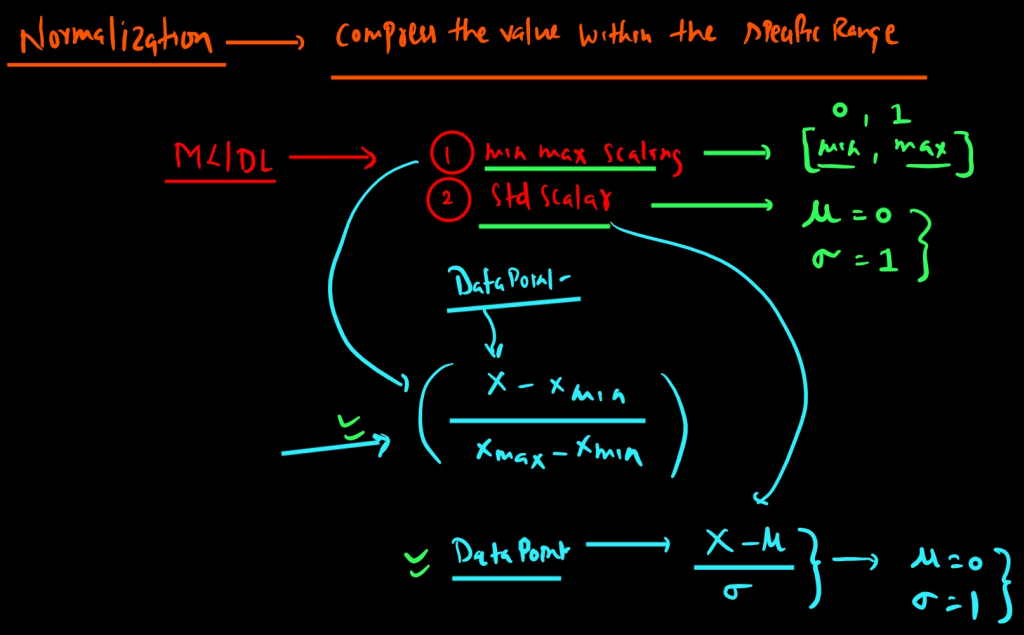
DL

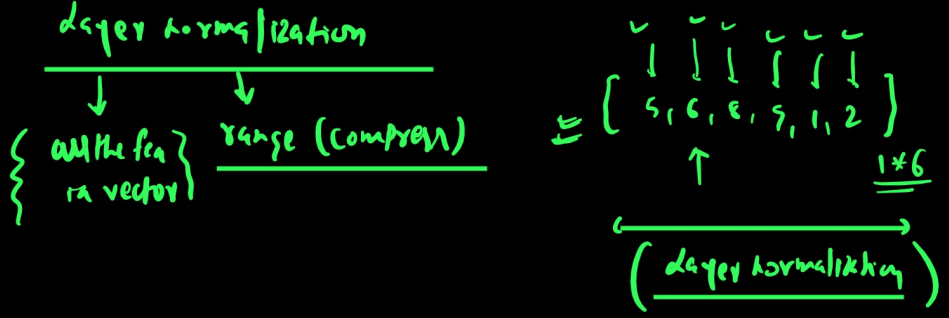
Adam W (optimizer)

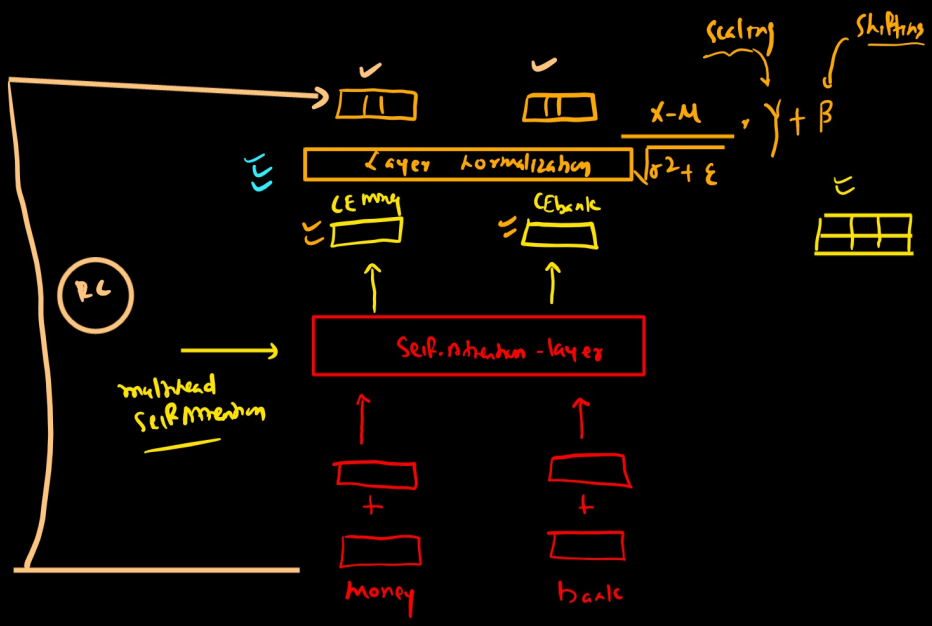
1. Stabilizes Training
Keeps values in a controlled range → prevents exploding or very large activations.

2. Improves Gradient Flow
Avoids vanishing/exploding gradients → helps deep transformers train smoothly.

3. Faster Convergence
Model learns faster because inputs to each layer are well-conditioned (consistent scale).

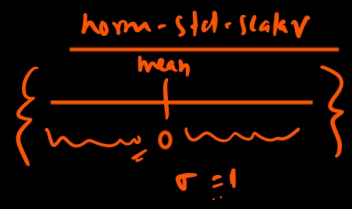






β gives the model the freedom to choose where the data should be centered.

Shifting Param

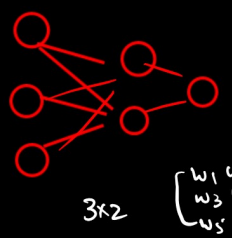


All the compt of encoder

Bze flow { Encoder to decoder
↓
Cross attention
multi attention }

1-2 hr

DL



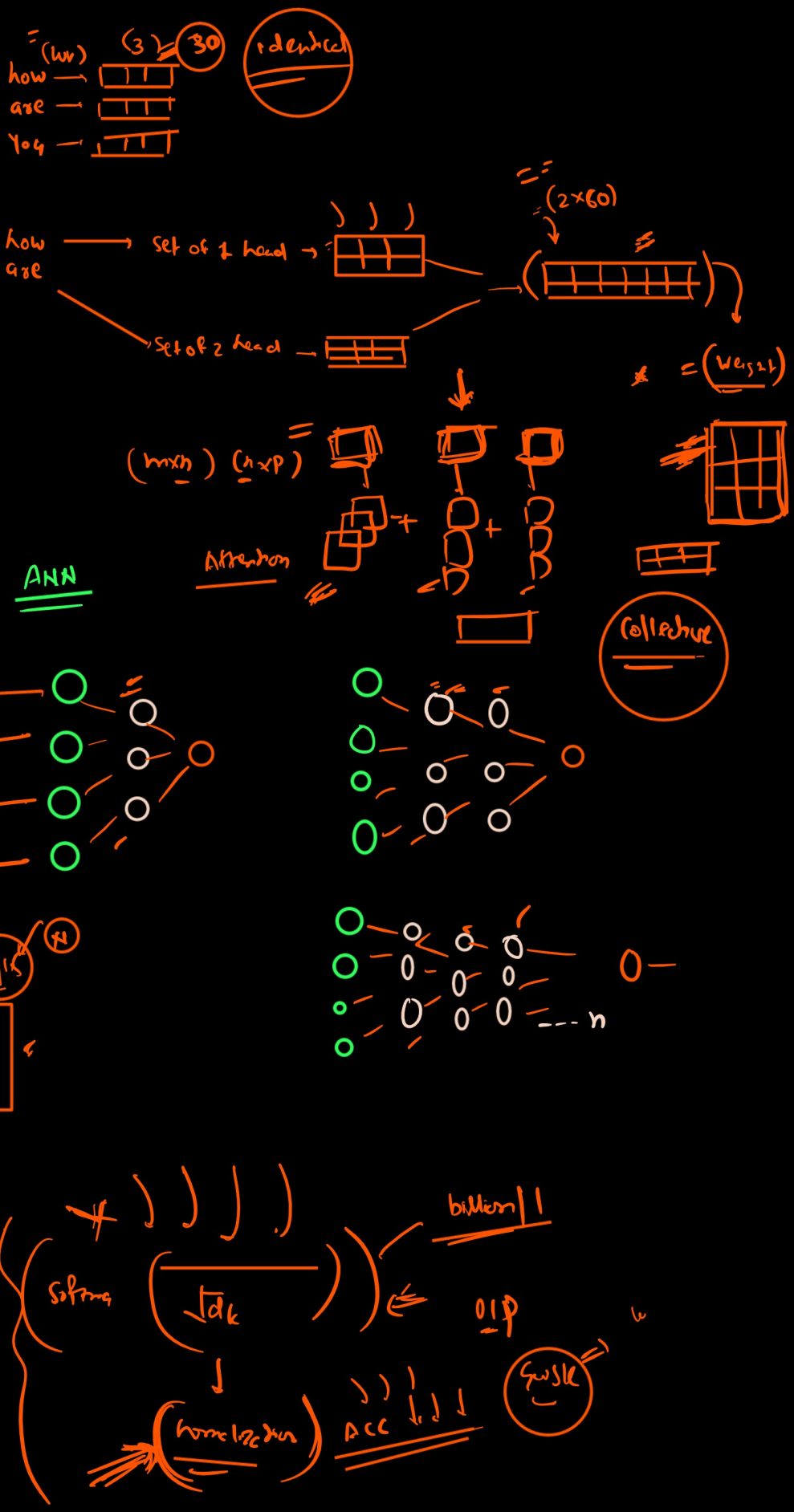
method ~ ① Random method

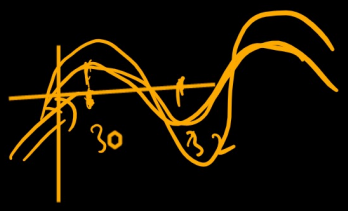
[ ]

during the train

②

③

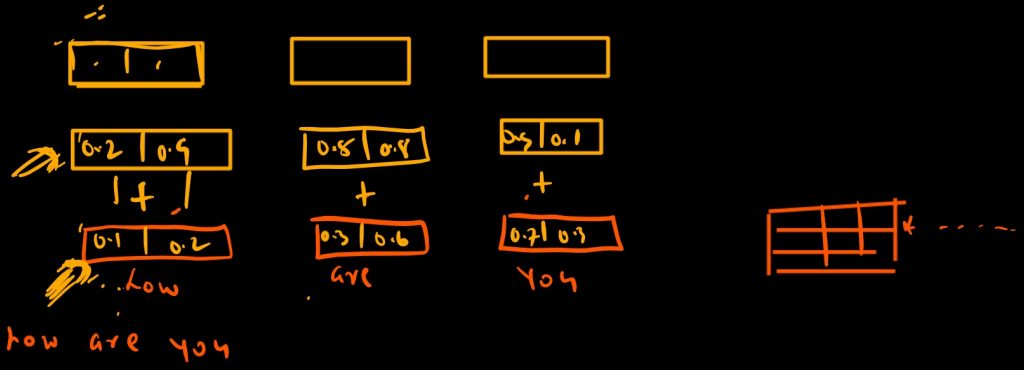

(w)
(3)
30
identical
how
are
you
how
are
set of 2 head
set of 2 head
(2×60)
(w,s,z)
(m×n) (n×p)
Attention
ANN
collective
hidden
10|15
N
30
n
billion||
Softmax
Idk
O|P
Normalization
Acc
Guess

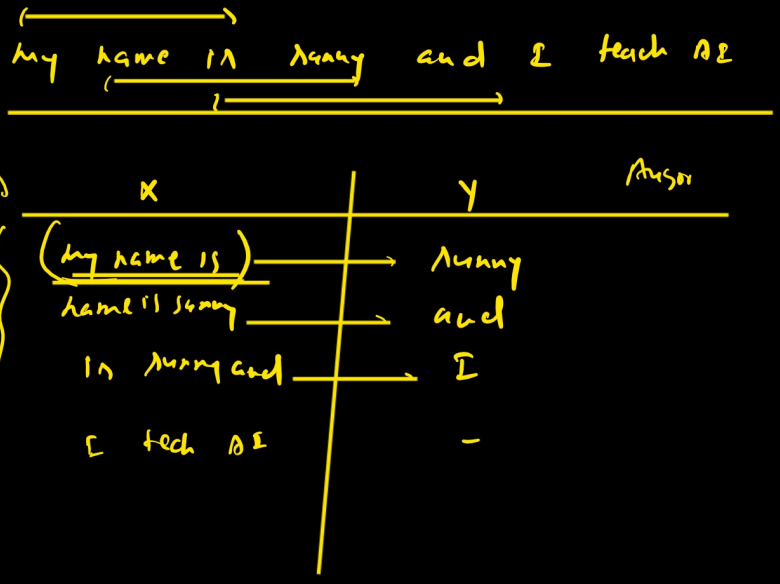


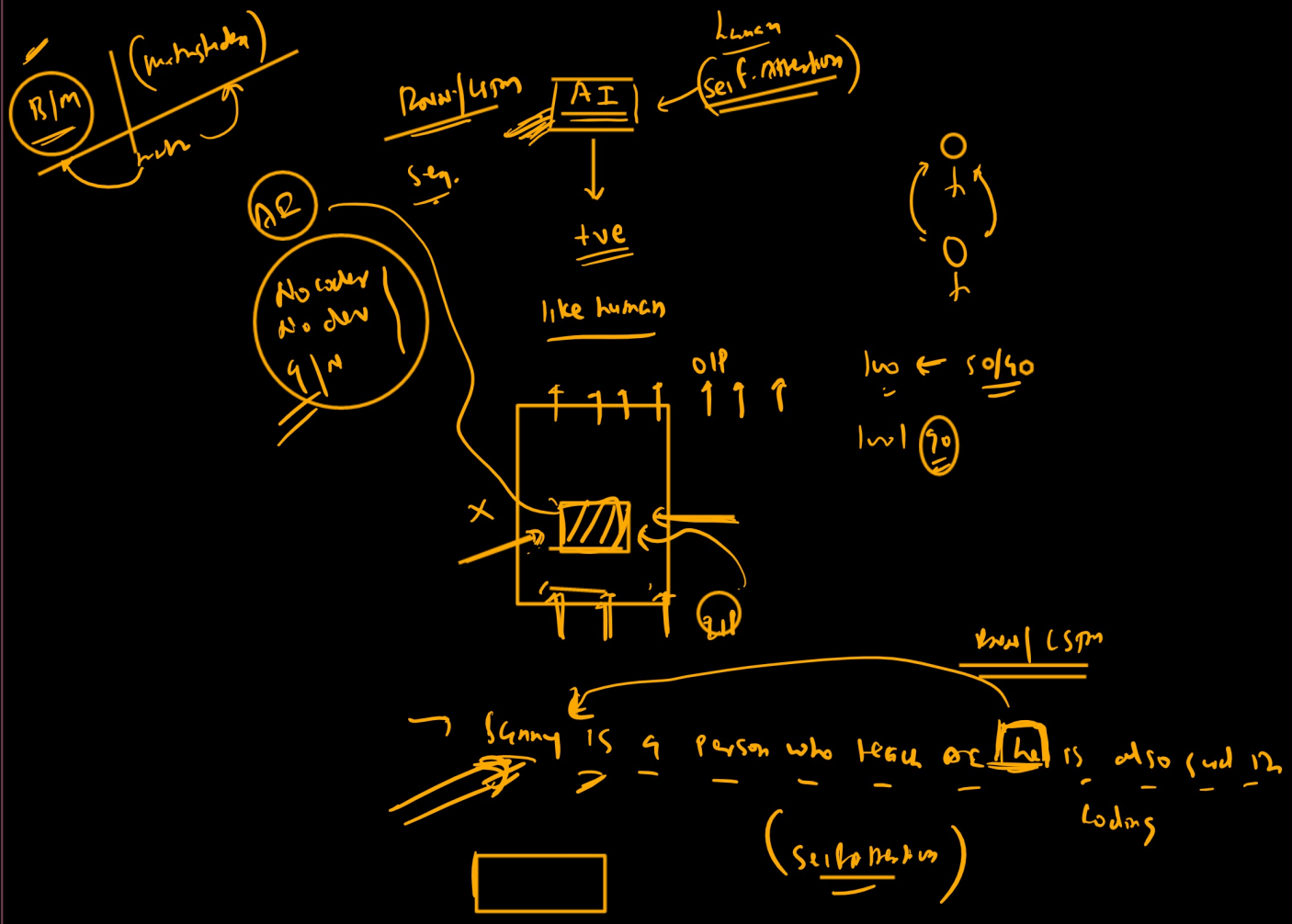
37 0.7 + 32 ← Mohan

75 0.5 + 30 ← Sunny

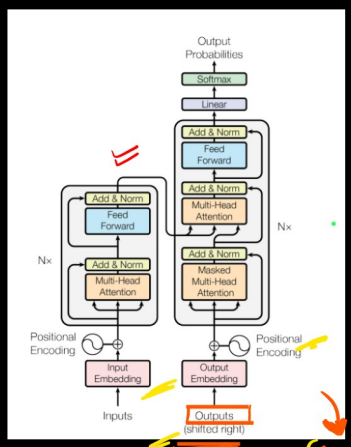
Mathematical



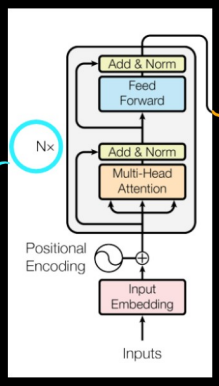


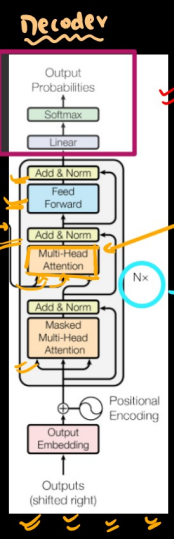

B/M
(machine)
Human
(self. attention)
AI
Seq.
+ve
like human
O/P
No color
No odor
Y/N
X
Sunny is a person who tells
Lodine

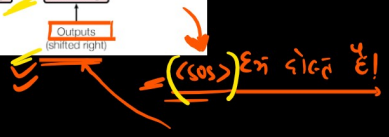
Decoder



MT







1. masked self-attention
2. Cross-Attention
3. Output head
4. Working / behaviour of the decoder

Originally the Transformer architecture was introduced for Neural Machine Translation (NMT) in the paper:

Attention Is All You Need

The original task was:

English → French
German → English
etc.

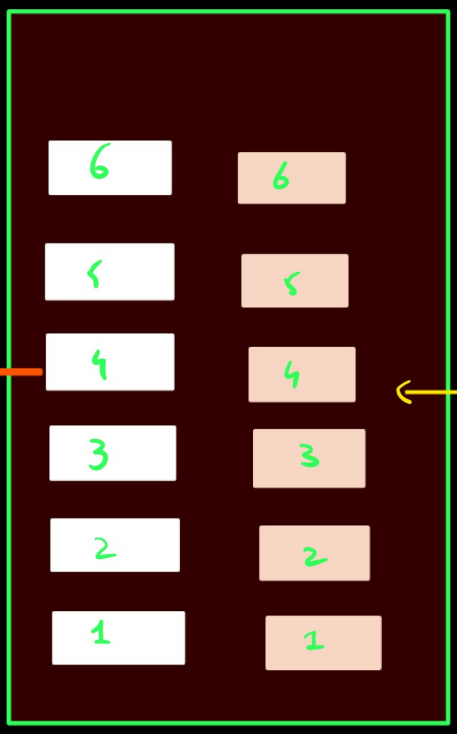
So the architecture had:

Encoder → reads source sentence
Decoder → generates translated target sentence word by word

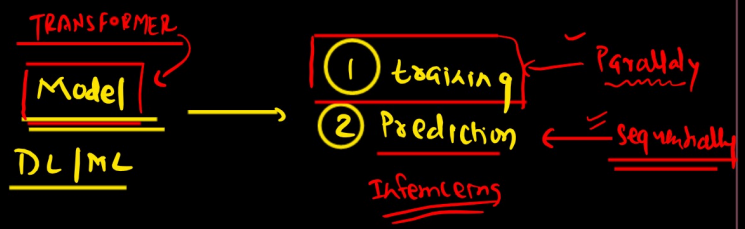
Transformer training is parallel, but inference is sequential

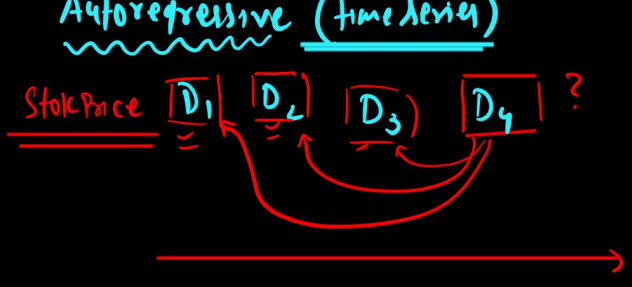
transformer decoder

→ it is a non autoregressive at the time of training
→ IT is autoregressive at the time of prediction


6
6
5
5
4
4
3
3
2
2
1
1
Encoder
Decoder
Nx=6

Encoder
FFNN
Self-Attention
Decoder
Output Head
FFNN
Cross-Attention
MASKED SA

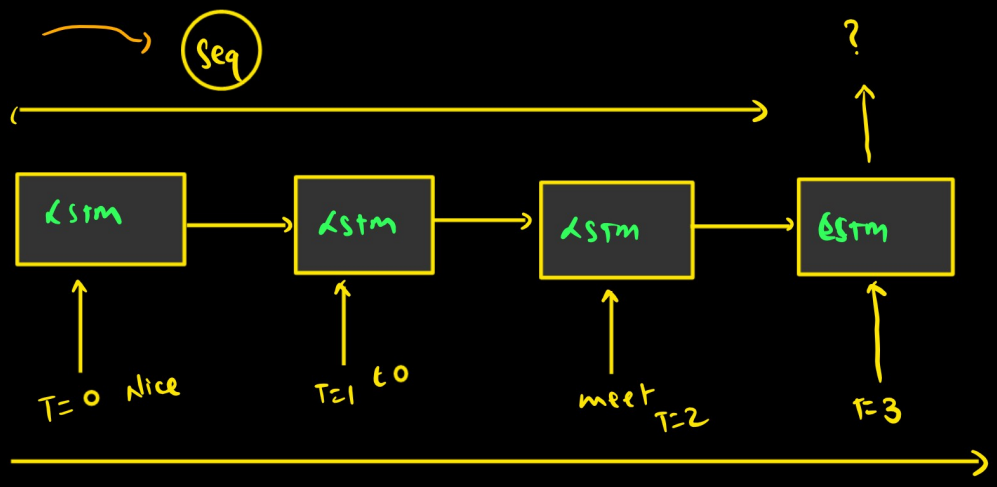

TRANSFORMER
Model
DL/ML
1 training
2 Prediction
Inferences
Parallely
sequentially


Autoregressive (time series)
Stock Price
D1
D2
D3
D4
?

RNN/LSTM → DL → 1 training, 2 Prediction/testing/Inferencing — Sequentially

TS

Nice to meet you ?



Seq → trains → slow

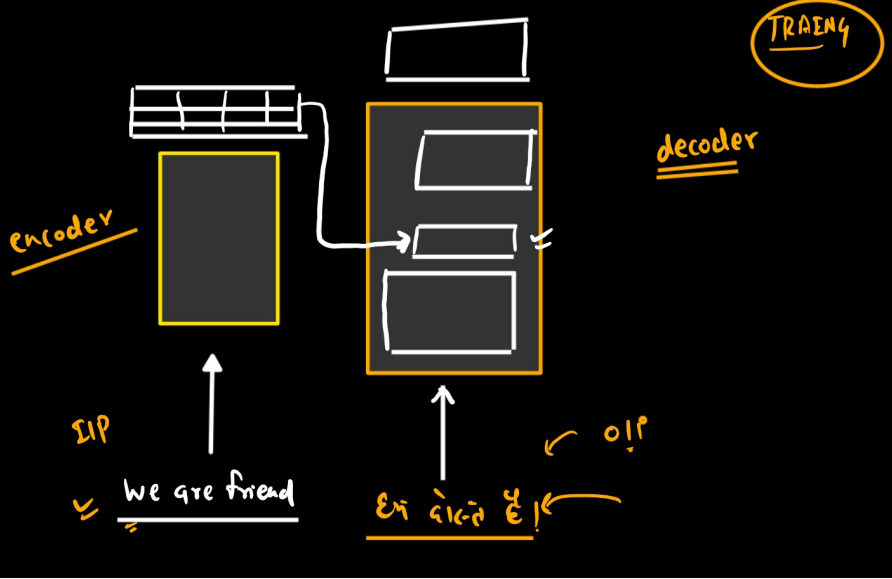
Transformer → Parallel → Fast — Less amt of data

Lot's and Lot's of data ← LLM

TRAINING → MT

We are friend | हम दोस्त हैं।

I/P O/P



ze_1^{Norm} ze_2^{Norm} ze_3^{Norm} ze_4^{Norm}

this output will be passed as input to feed forward Network (NN)

Layer - normalization

ze_1' ze_2' ze_3' ze_4'

$z_1'^{Norm}$ $z_2'^{Norm}$ $z_3'^{Norm}$ $z_4'^{Norm}$

ze^1 ze^2 ze^3 ze^4

(Residual connection)

In CROSS Attention

Q O|P

K, V I|P

this input

Input

CROSS - Attention - Layer

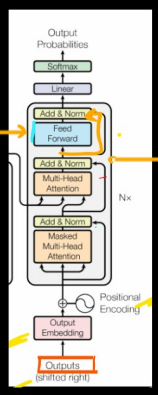
We are Friend

This is coming from encoder

$z_1'^{Norm}$ $z_2'^{Norm}$ $z_3'^{Norm}$ $z_4'^{Norm}$ this input

LAYER - NORMALIZATION

NN



z_1' z_2' z_3' z_4'

x_1 x_2 x_3 x_4

\+

z_1 z_2 z_3 z_4

SKIP-CONNECTION

Residual connection

?

MASKD SELF Attention Layer

x_1 x_2 x_3 x_4

Final vector

PE1 PE2 PE3 PE4

Positional encoding

$+e_1$ $+e_2$ $+e_3$ $+e_4$

Embedding Layer

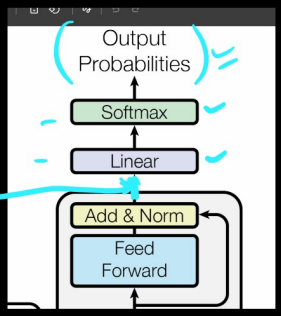
Word2vec - Lookup (Original Research Paper Dim-512)

<SOS> <en> <दोस्त> <ई>

tokenization

हम दोस्त हैं।

Final Prediction Layer



(4X512)

Y_1'Norm Y_2'Norm Y_3'Norm Y_4'Norm

Layer Normalization Layer

Y_1' Y_2' Y_3' Y_4'

ze_1'Norm ze_2'Norm ze_3'Norm ze_4'Norm

Residual connection

Y_1 Y_2 Y_3 Y_4

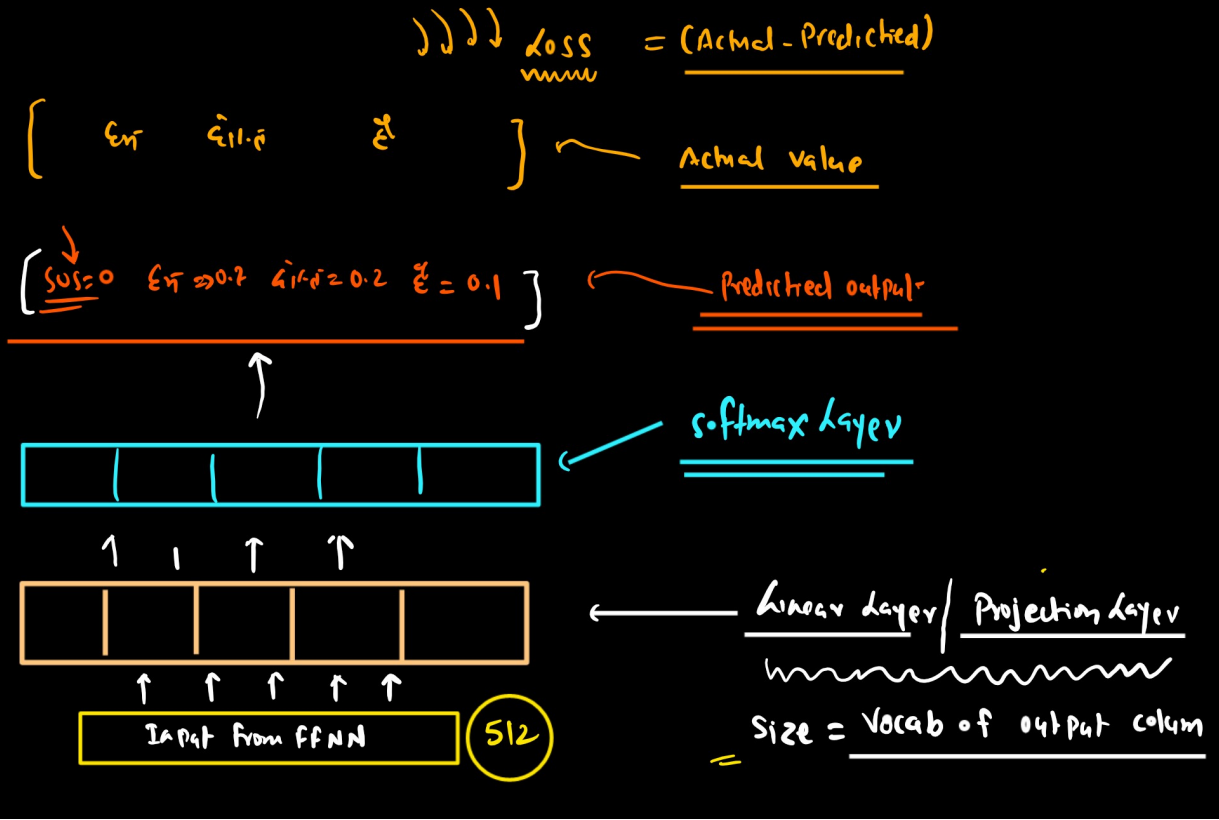
FFNN

H1 2048

Original Paper Reasearch

(4X512)

ze_1'Norm ze_2'Norm ze_3'Norm ze_4'Norm


Loss = (Actual - Predicted)
Actual value
Predicted output
softmax layer
Linear layer / Projection layer
Size = Vocab of output column
Input from FFNN
512

Autoregssive Inference